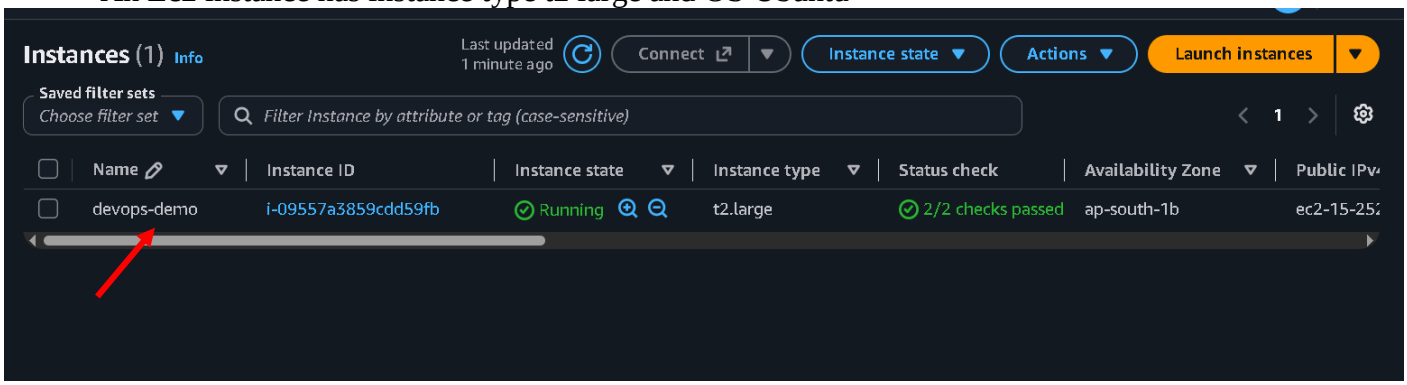
Ultimate DevOps Project Implementation

Summary: -

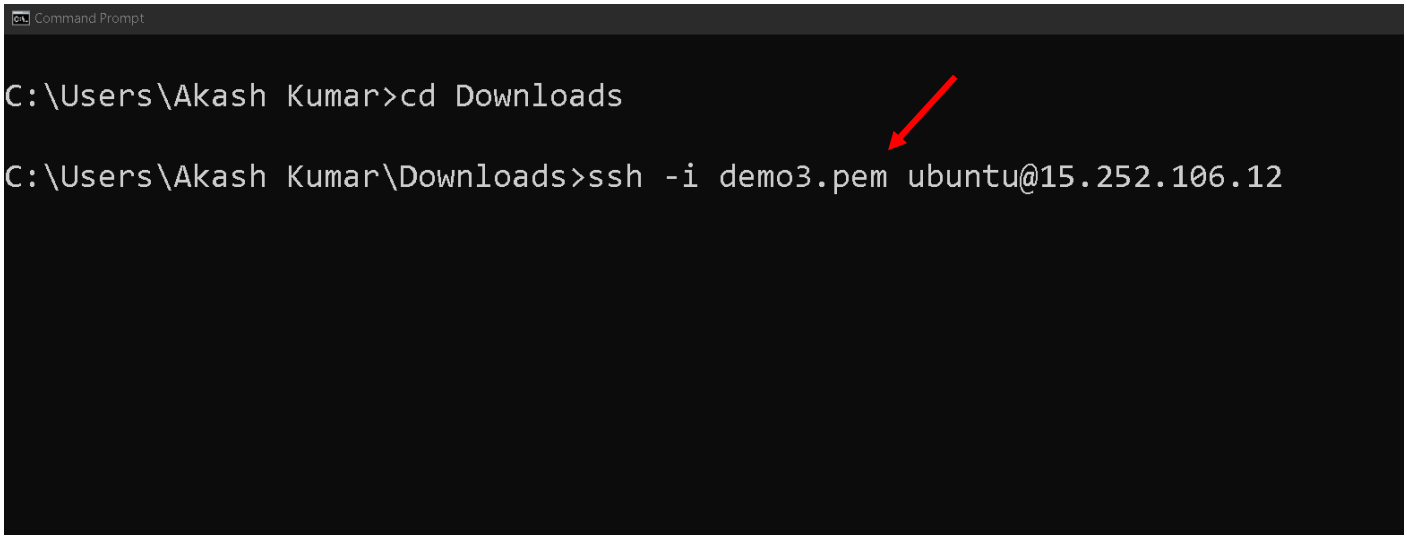
I created AWS infrastructure using Terraform, deployed a microservices application on Kubernetes, and set up CI/CD automation using GitHub Actions and Docker.

Installation and Prerequisites: -

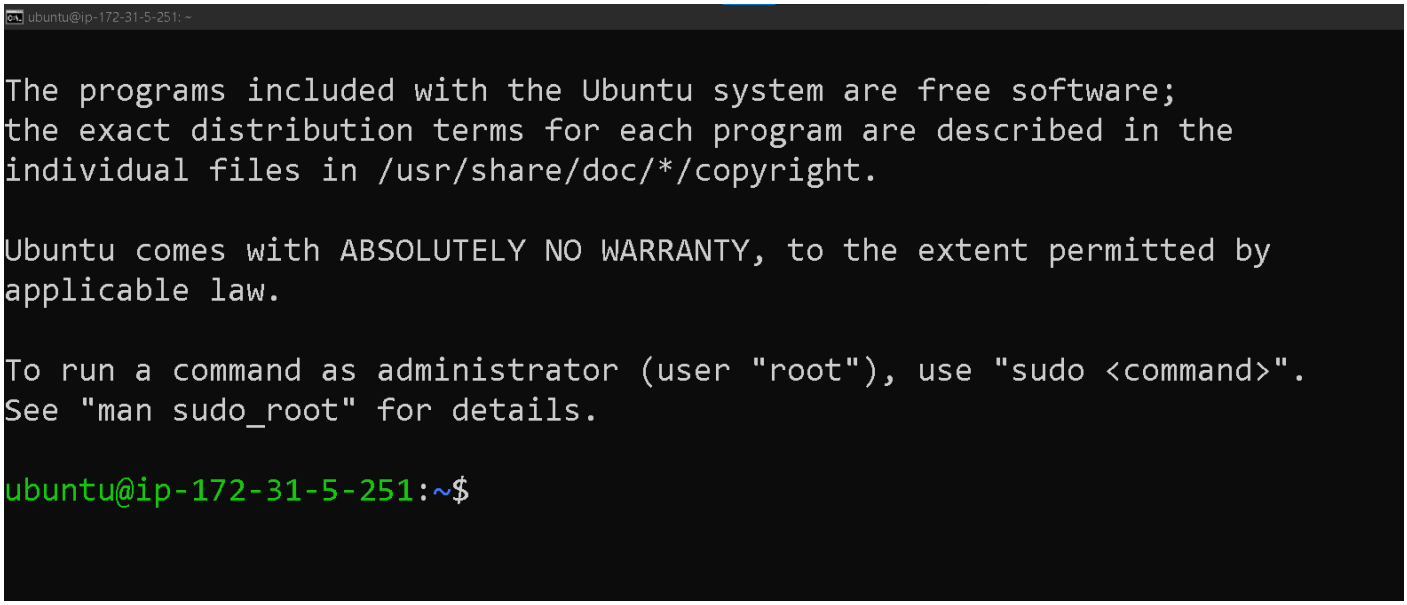
- Create your AWS account.
- And then create IAM user with administrative access.
- And then login with IAM user and create EC2 instance.
- An Ec2 instance has instance type t2 large and OS Ubuntu



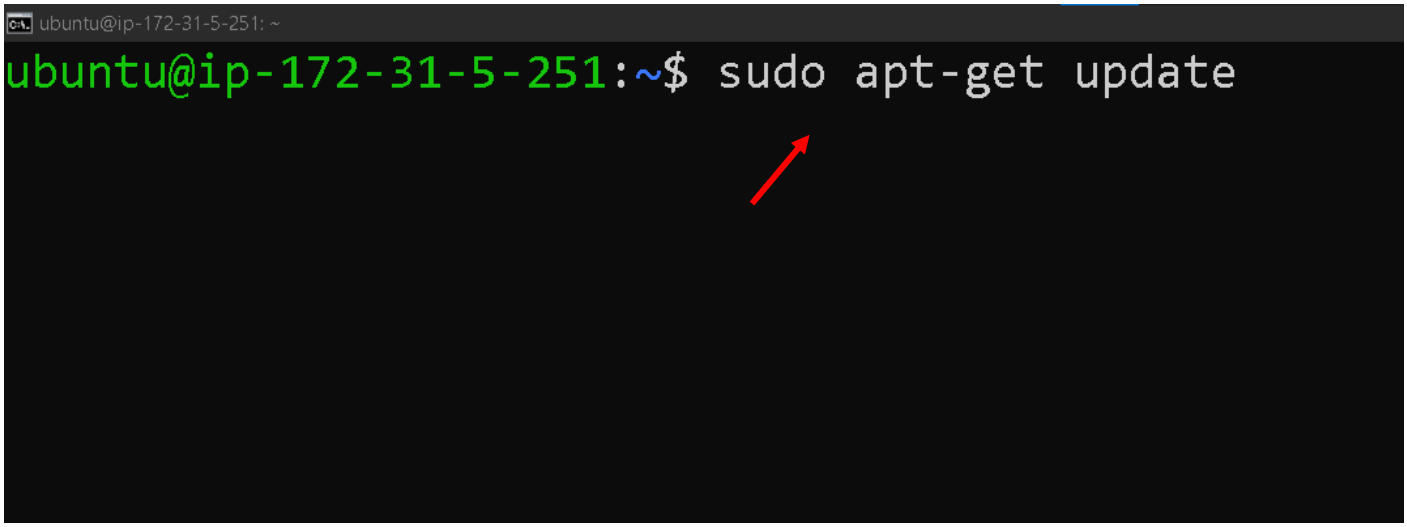
- Our instance is created now open command prompt



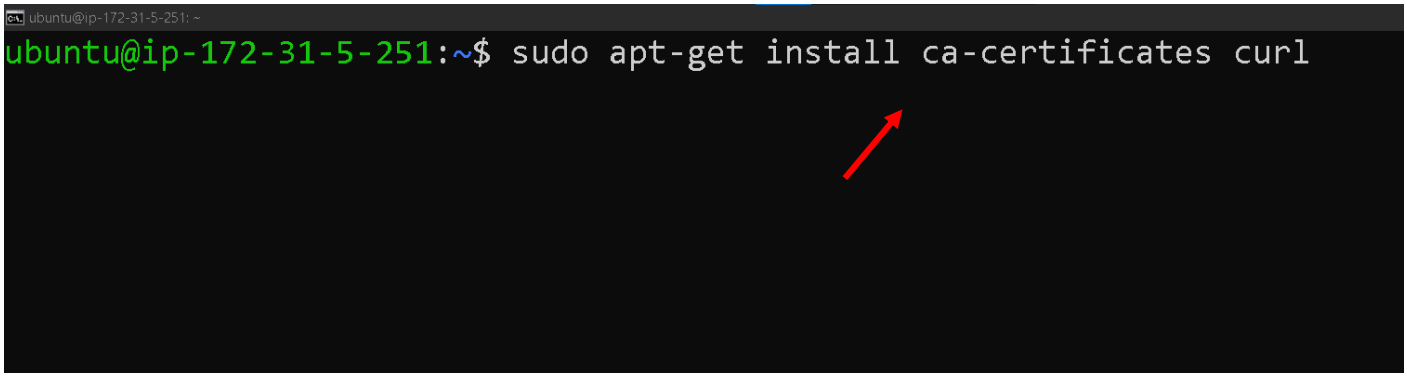
- Now go to Folder where you download Key.
- Then type ssh -i keyname ubuntu@ip address



- Now we connected to our Ec2 instance.
- Now Let's install docker to our EC2



- sudo apt-get update



- sudo apt-get install ca-certificates curl

```
Reading state information... Done
ca-certificates is already the newest version (20260816ubuntu0.24.04.1).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.16).
0 upgraded, 0 newly installed, 0 to remove and 17 not upgraded.
ubuntu@ip-172-31-5-251:~$ sudo install -m 0755 -d /etc/apt/keyrings
```

- sudo install -m 0755 -d /etc/apt/keyrings

```
ubuntu@ip-172-31-5-251:~$ sudo apt-get install ca-certificates curl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20260816ubuntu0.24.04.1).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.16).
0 upgraded, 0 newly installed, 0 to remove and 17 not upgraded.
ubuntu@ip-172-31-5-251:~$ sudo install -m 0755 -d /etc/apt/keyrings
ubuntu@ip-172-31-5-251:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
```

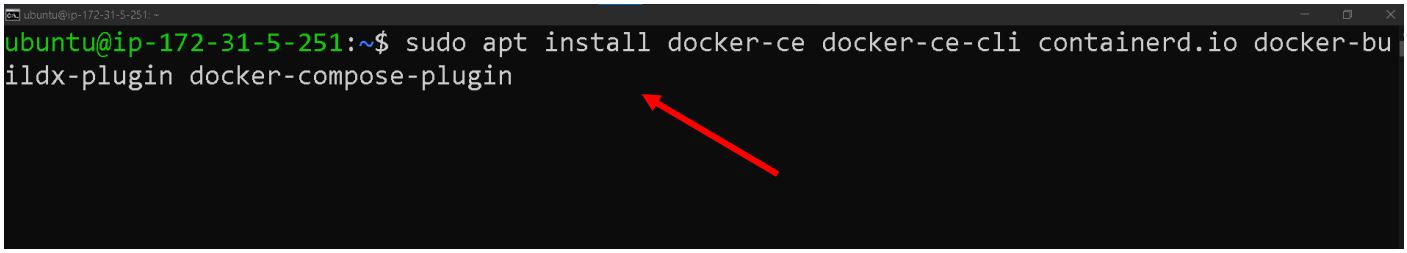
- sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc

```
/etc/apt/keyrings/docker.asc
ubuntu@ip-172-31-5-251:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
```

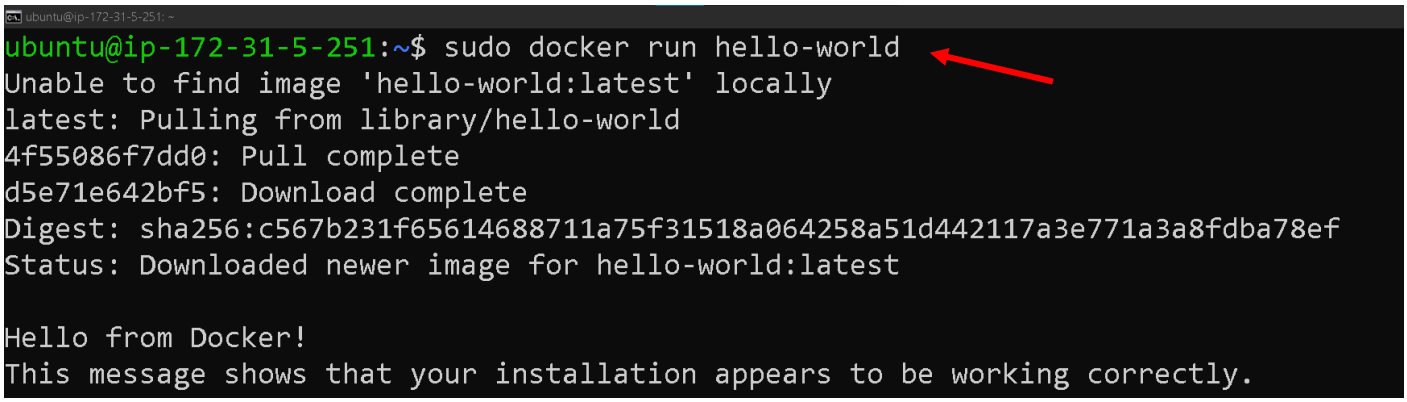
- sudo chmod a+r /etc/apt/keyrings/docker.asc

```
ubuntu@ip-172-31-5-251:~$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
> Types: deb
> URIs: https://download.docker.com/linux/ubuntu
> Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
> Components: stable
> Architectures: $(dpkg --print-architecture)
> Signed-By: /etc/apt/keyrings/docker.asc
> EOF
```

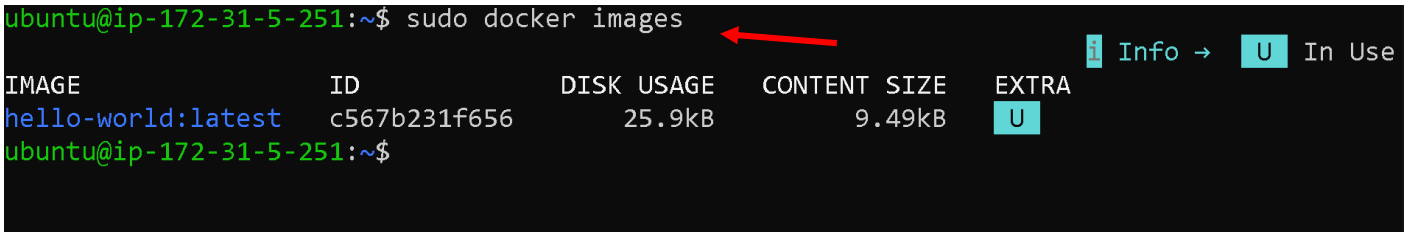
- it has provided in resource.



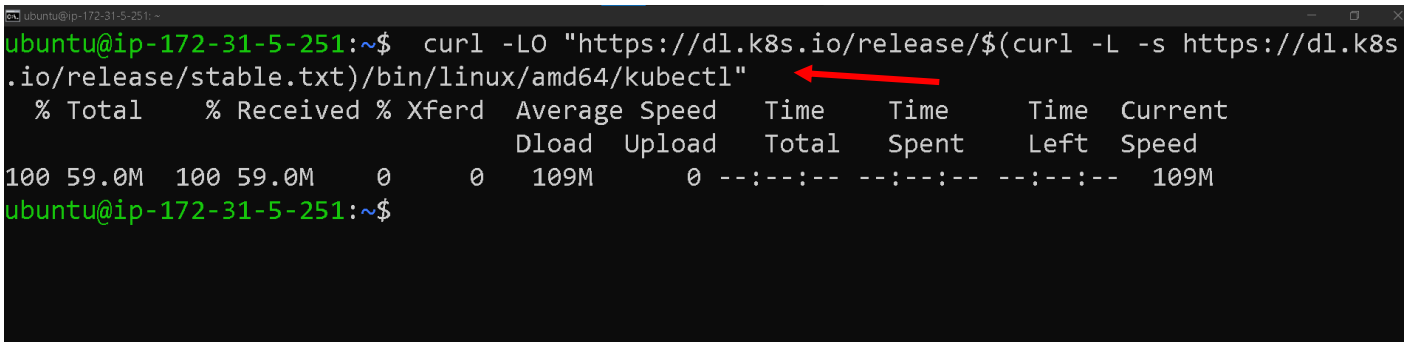
- sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin



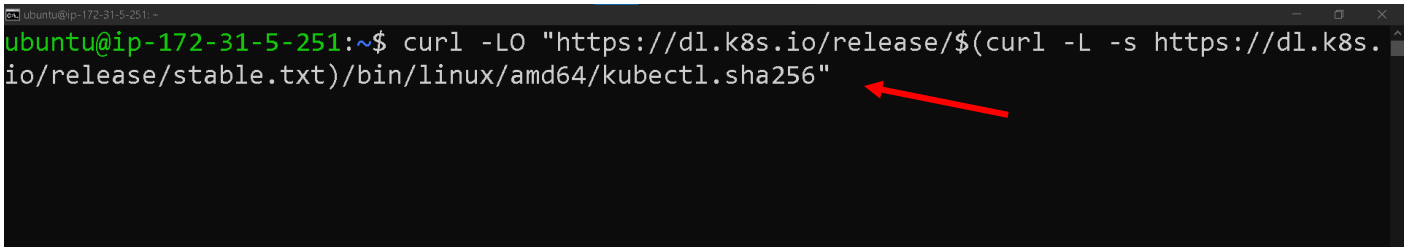
- sudo docker run hello-world



- Now our Docker is perfectly install.
- Now we install kubectl.



- curl -LO https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl



- curl -LO https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256
- to install kubectl checksum file.

```
ubuntu@ip-172-31-5-251:~$ echo "$(cat kubectl.sha256)  kubectl" | sha256sum --check
kubectl: OK
ubuntu@ip-172-31-5-251:~$
```

- echo "$(cat kubectl.sha256) kubectl" | sha256sum –check

```
ubuntu@ip-172-31-5-251:~$ ls
kubectl  kubectl.sha256
ubuntu@ip-172-31-5-251:~$ sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kube
ctl
```

- install the kubectl.
- sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

```
ubuntu@ip-172-31-5-251:~$ kubectl version --client
Client Version: v1.37.2
Kustomize Version: v5.8.1
ubuntu@ip-172-31-5-251:~$
```

- Now check its version.
- kubectl version –client
- Now we install terraform

```
ubuntu@ip-172-31-5-251:~$ wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --de
armor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
--2026-11-02 12:22:09--  https://apt.releases.hashicorp.com/gpg
sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraformResolving apt.releases.hashicorp.com (apt.rel
eases.hashicorp.com)... 13.225.5.7, 13.225.5.13, 13.225.5.32, ...
Connecting to apt.releases.hashicorp.com (apt.releases.hashicorp.com)|13.225.5.7|:443... c
onnected.
HTTP request sent, awaiting response... 200 OK
Length: 1725 (1.7K) [application/x-www-form-urlencoded]
Saving to: ‘STDOUT’

-                   100%[=========================>]   1.68K  --.-KB/s    in 0s

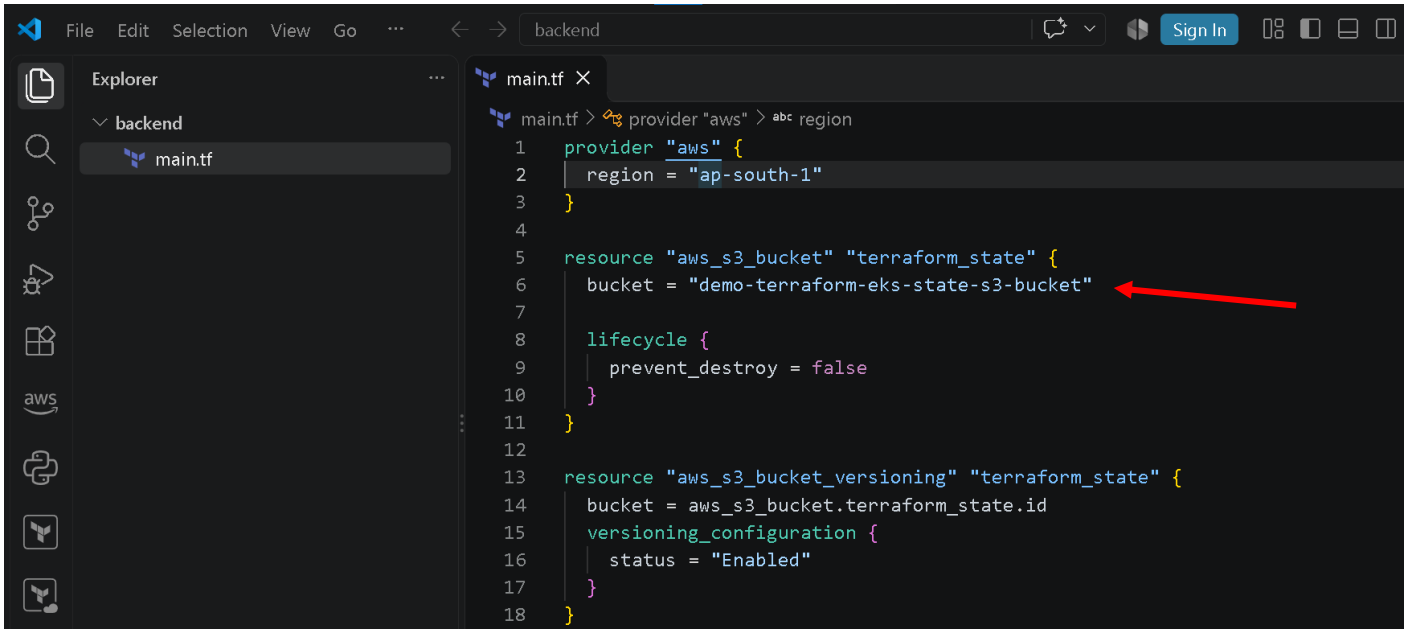
2026-11-02 12:22:09 (589 MB/s) - written to stdout [1725/1725]
```

- Install manually by Link

```
ubuntu@ip-172-31-5-251: ~
ubuntu@ip-172-31-5-251:~$ terraform --version
Terraform v1.17.1
on linux_amd64

ubuntu@ip-172-31-5-251:~$
```

- Now see its install.
- Now Create S3 bucket and Dynamo DB using Terraform.
- Open your Vs Code.

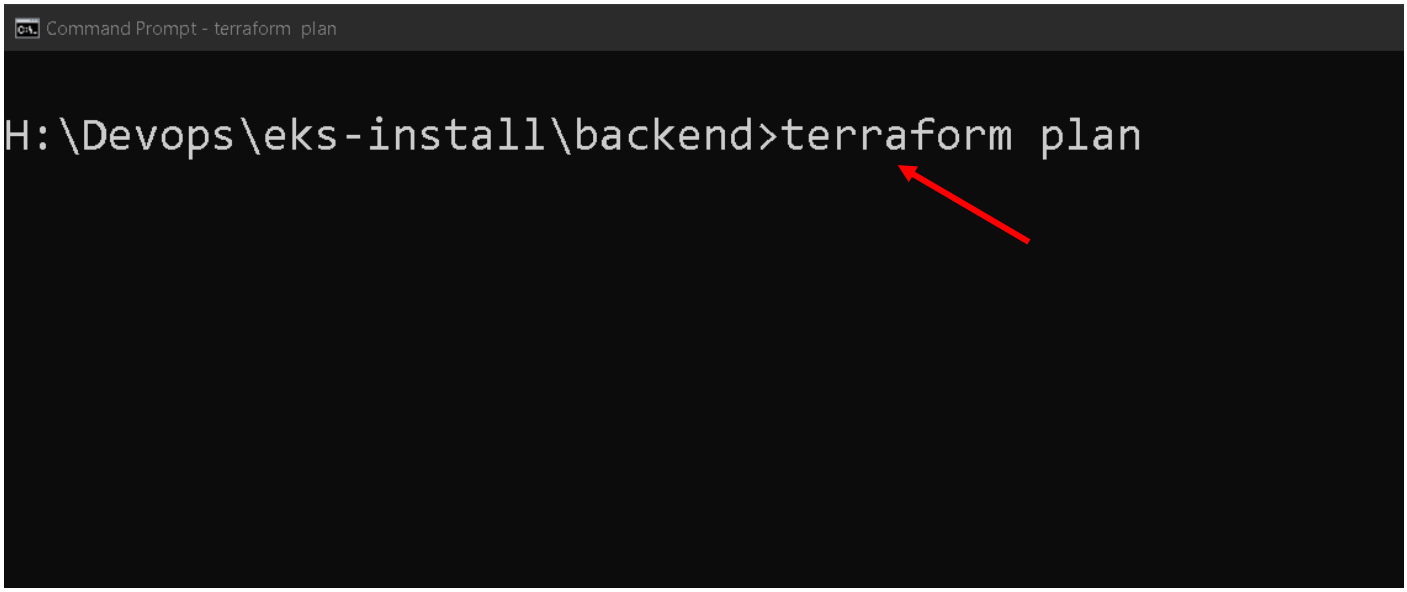


- Here we created our S3 bucket and dynamo Db
- Now open Command prompt.
- Firstly install terraform in your system.

```
H:\Devops\eks-install\backend>terraform init
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.71.0...
```

- Now go to the file directory.
- And then type terraform init



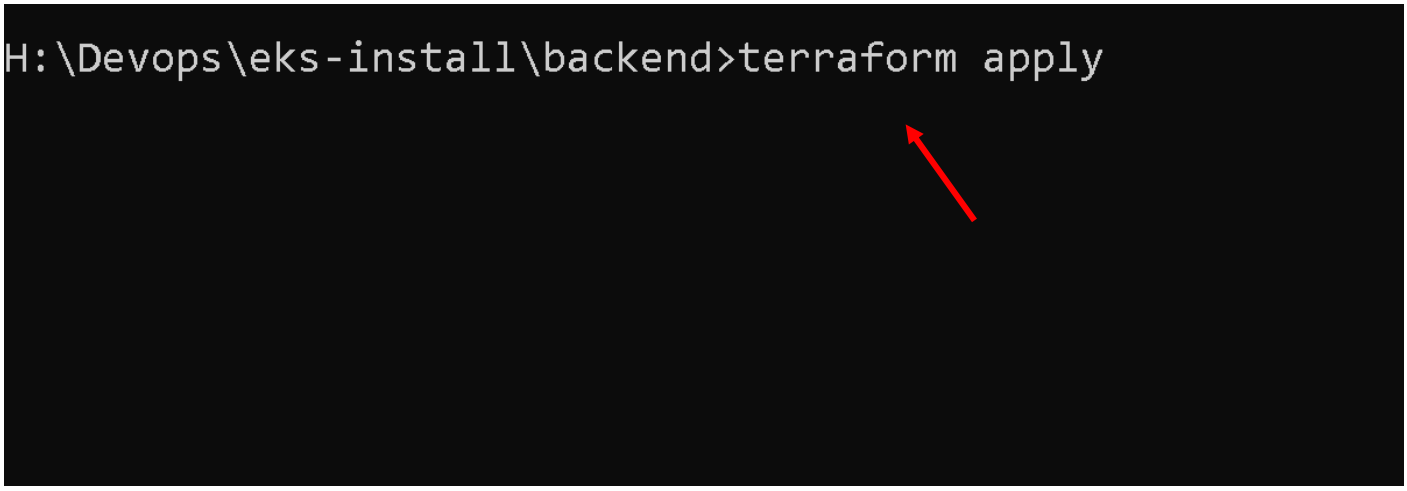
- Now check both the resource is created or not.

```
Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
  + create

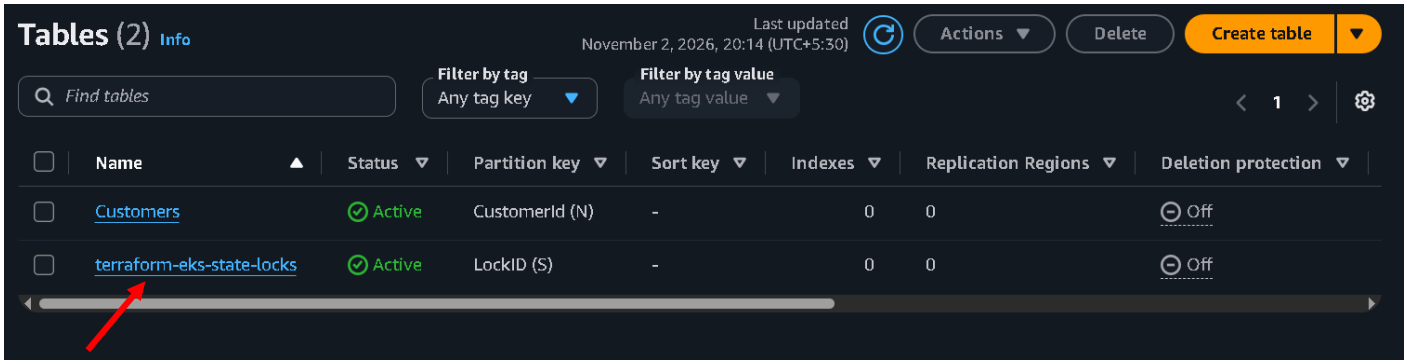
Terraform will perform the following actions:

  # aws_dynamodb_table.terraform_locks will be created
  + resource "aws_dynamodb_table" "terraform_locks" {
      + arn              = (known after apply)
      + billing_mode     = "PAY_PER_REQUEST"
      + hash_key         = "LockID"
      + id               = (known after apply)
      + name             = "terraform-eks-state-locks"
      + read_capacity    = (known after apply)
```

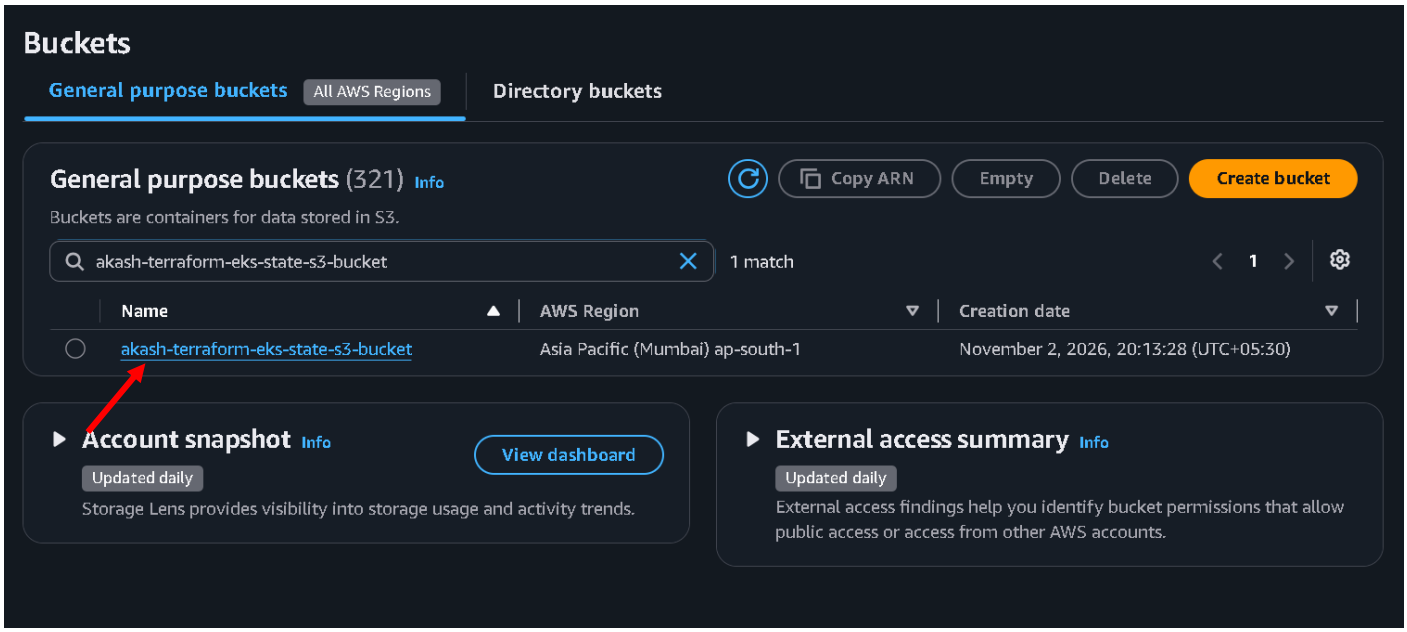
- Check it executed or not
- terraform plan



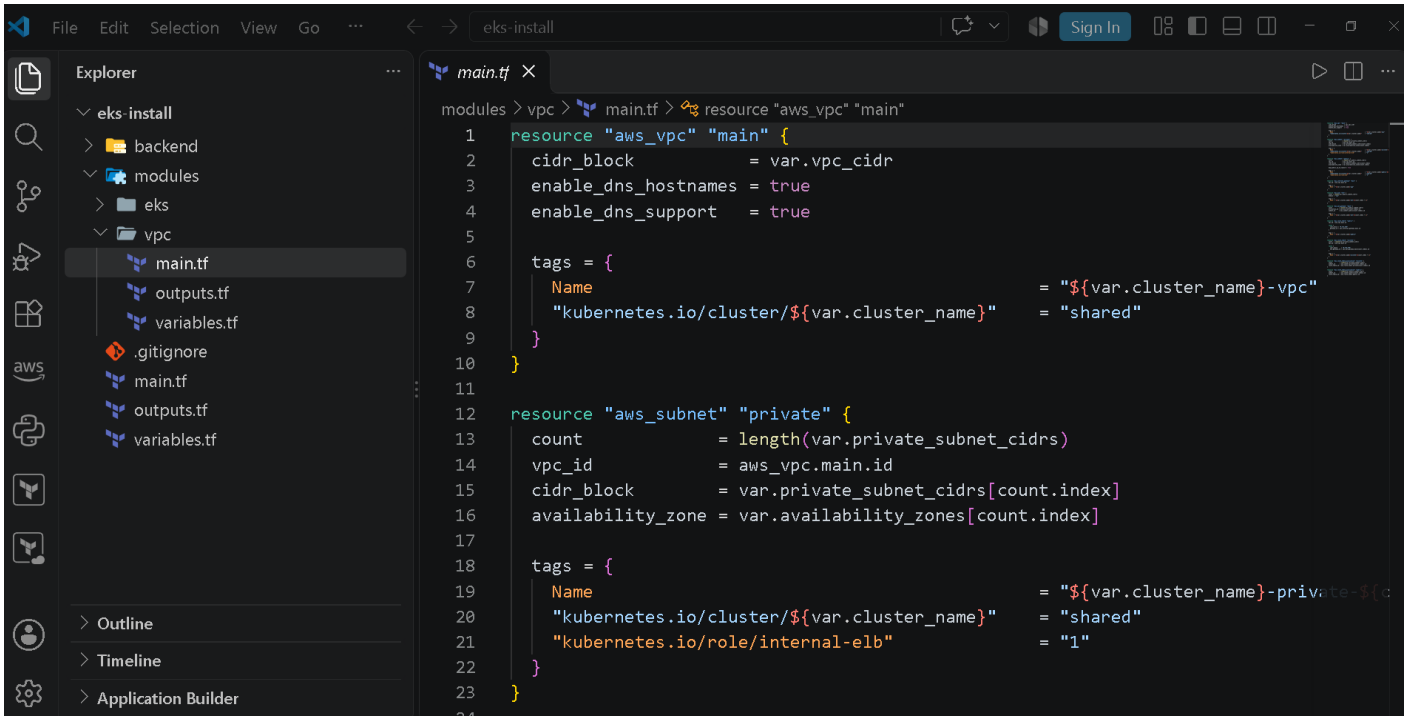
- Now apply it terraform apply
- Now go and check on Aws console both are created or not



- Dynamo DB is created.

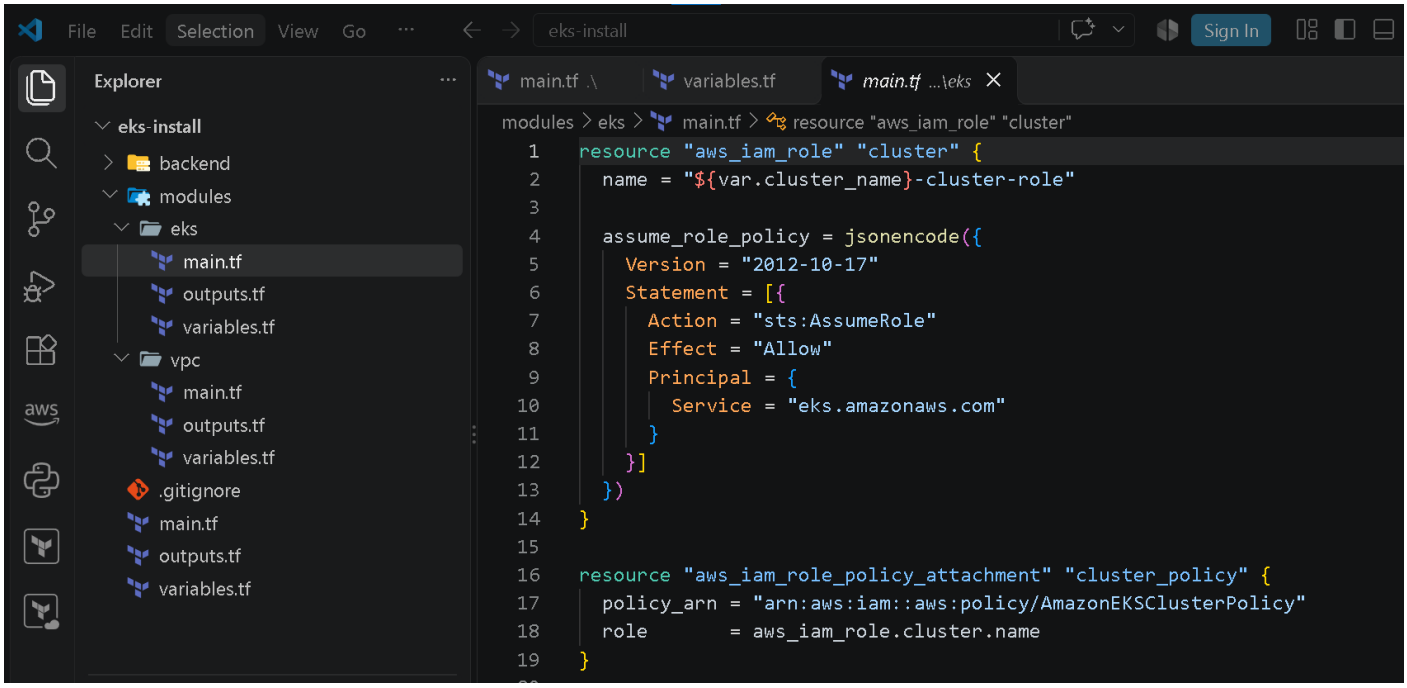


- S3 bucket is also created.
- Now We create our Vpc and eks.

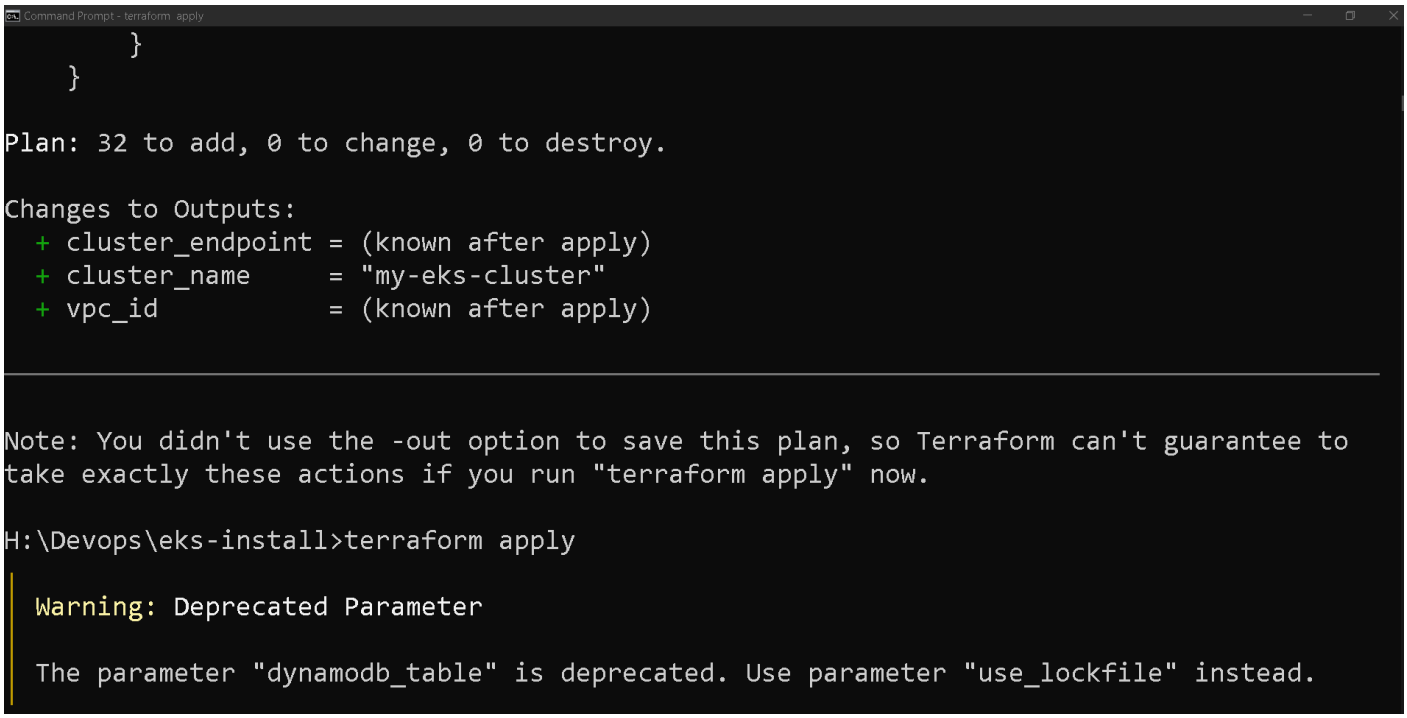


- We created a module folder and in module folder we have two folder one is VPC and second one is eks
- In vpc we have there terraform file main, variable and output.

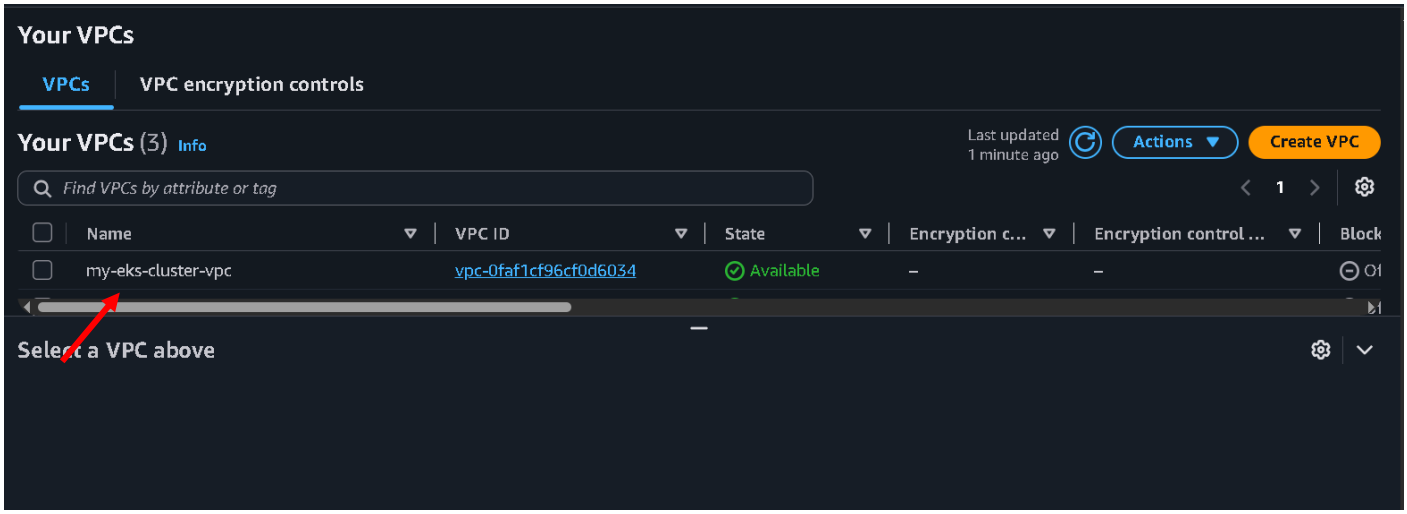
- In main I write vpc resource, public subnet, private subnet, nat gateway, internet gateway and route table resource.
- In public subnet should be connected to internet gateway.
- And the private subnet should we connected to nat gateway.



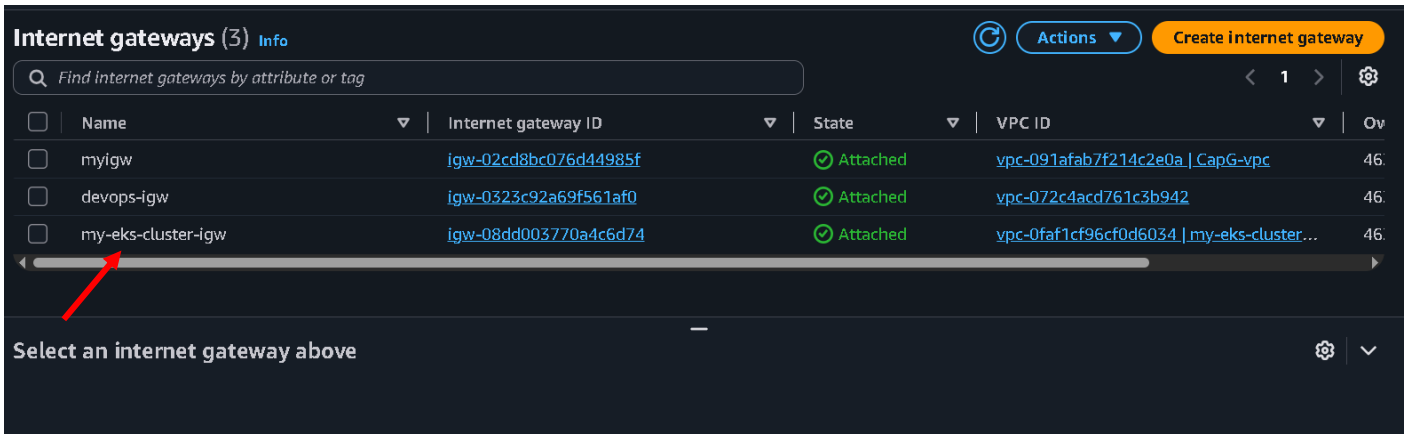
- In eks we create a 2 IAM role one for cluster and one for node.
- And also associate policy to both IAM role.
- Now implements this go to your command prompt



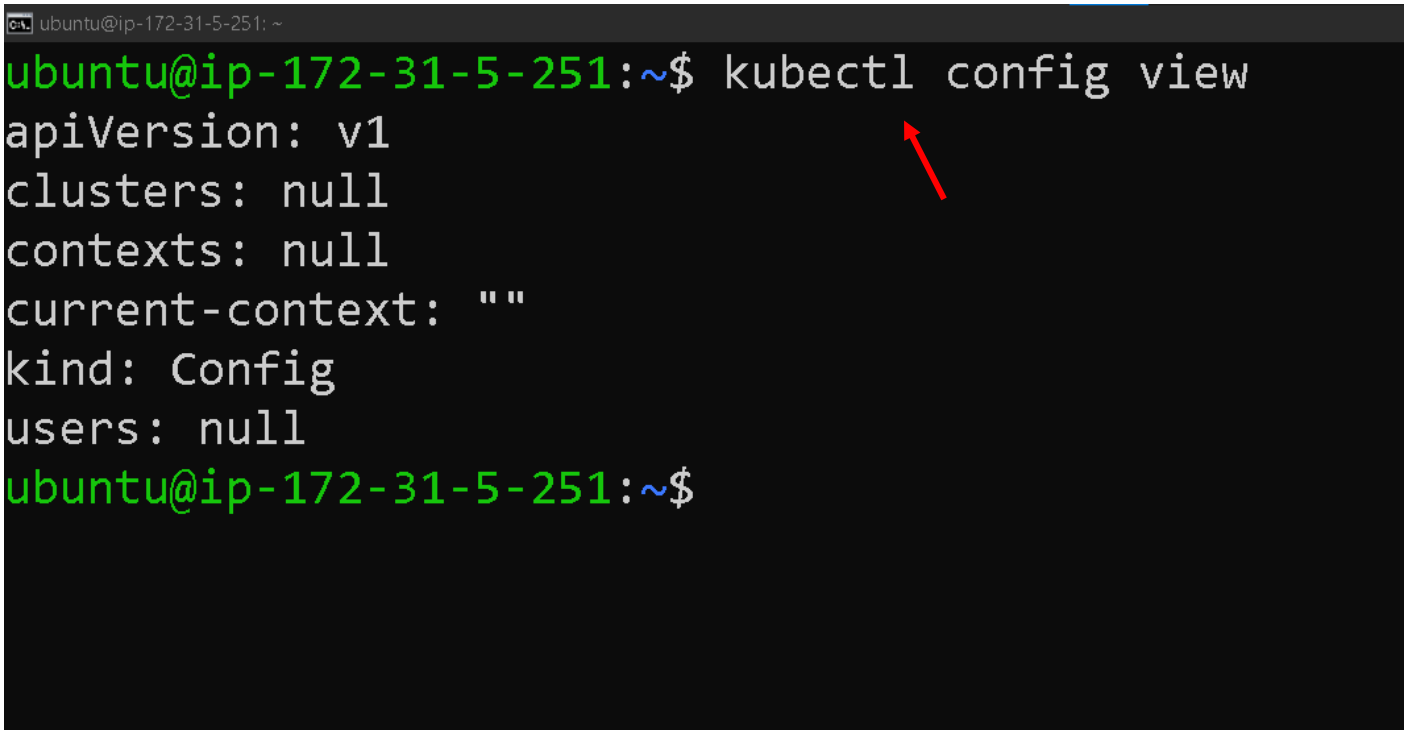
- Basically three command we perform one by one 1st terraform init 2nd terraform plan and 3rd terraform apply.
- Now wait it will take time.
- When completed go to Aws Console and check its created or not.



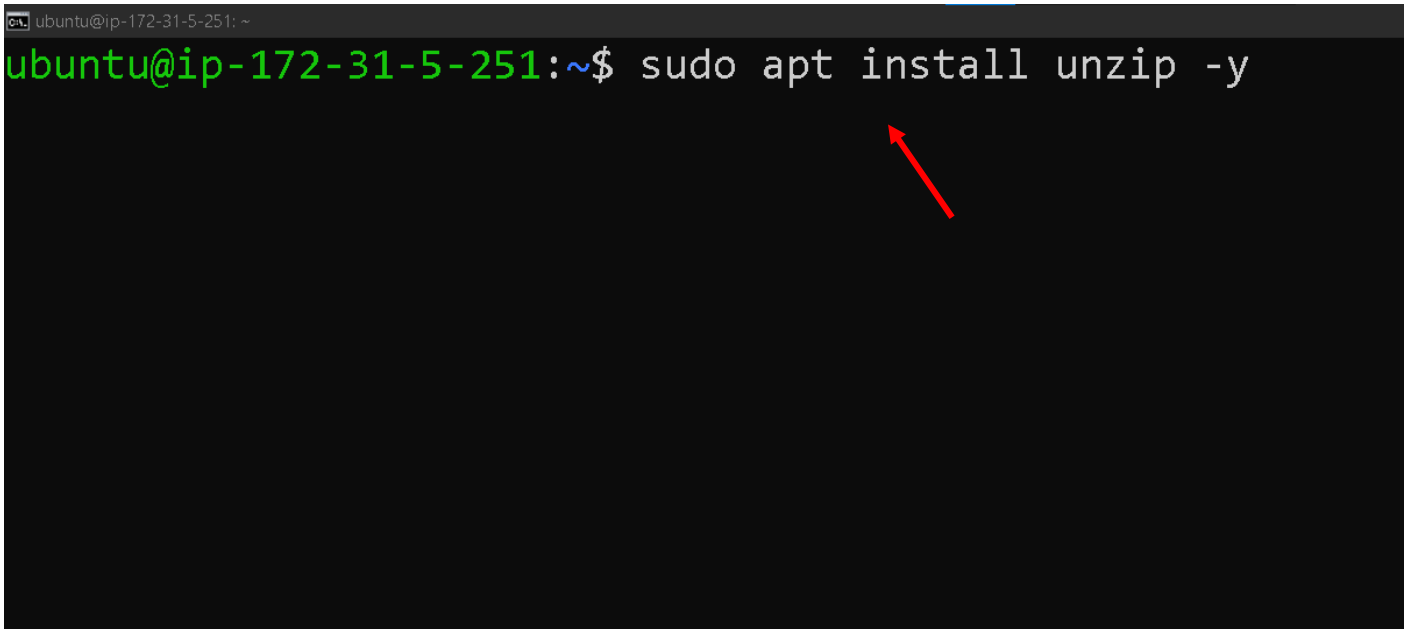
- Now see Vpc is created .



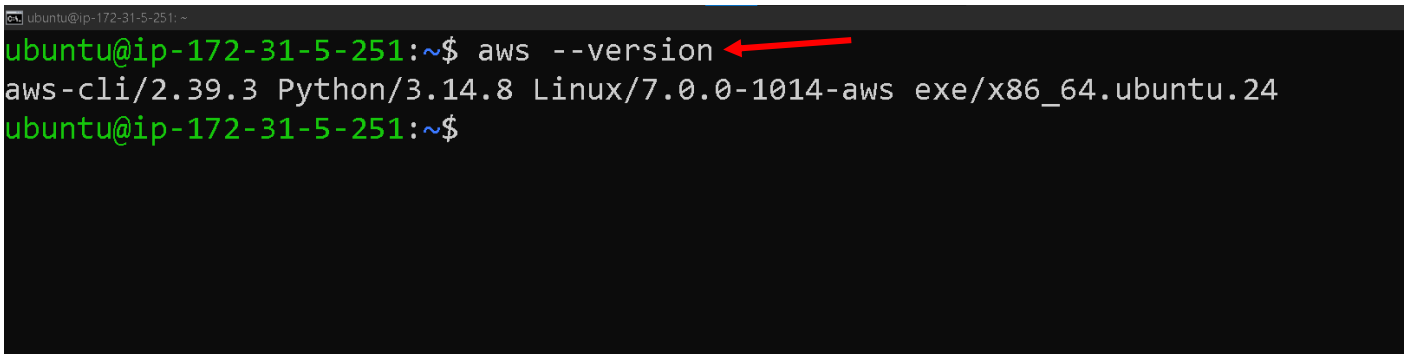
- Internet gateway is also created now see all things are created.
- Now we need to access our eks-cluster by EC2-instance



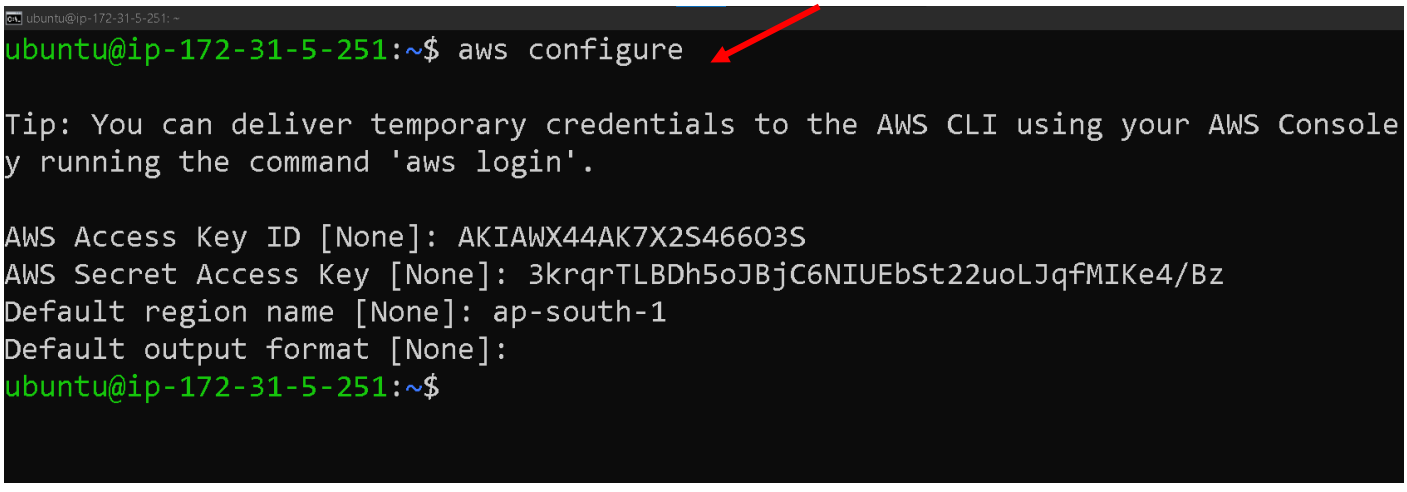
- Currently in our kubectl have no cluster.
- Firstly we install aws cli in our Ec2



- sudo apt install unzip –y
- now install Aws cli.
- Code get to install [Link]



- aws --version
- to confirm it install or not.



- now after installation you need to configure aws in it.

```
ubuntu@ip-172-31-5-251:~$ aws eks update-kubeconfig --region ap-south-1 --name my-eks-clus
ter
```

- aws eks update-kubeconfig --region region-code --name clusterName

```
ubuntu@ip-172-31-5-251:~$ kubectl config view
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://A271FB8EE919A407B8A67813B1C1F6C6.gr7.ap-south-1.eks.amazonaws.com
  name: arn:aws:eks:ap-south-1:463646775279:cluster/my-eks-cluster
contexts:
- context:
    cluster: arn:aws:eks:ap-south-1:463646775279:cluster/my-eks-cluster
    user: arn:aws:eks:ap-south-1:463646775279:cluster/my-eks-cluster
  name: arn:aws:eks:ap-south-1:463646775279:cluster/my-eks-cluster
current-context: arn:aws:eks:ap-south-1:463646775279:cluster/my-eks-cluster
kind: Config
users:
- name: arn:aws:eks:ap-south-1:463646775279:cluster/my-eks-cluster
```

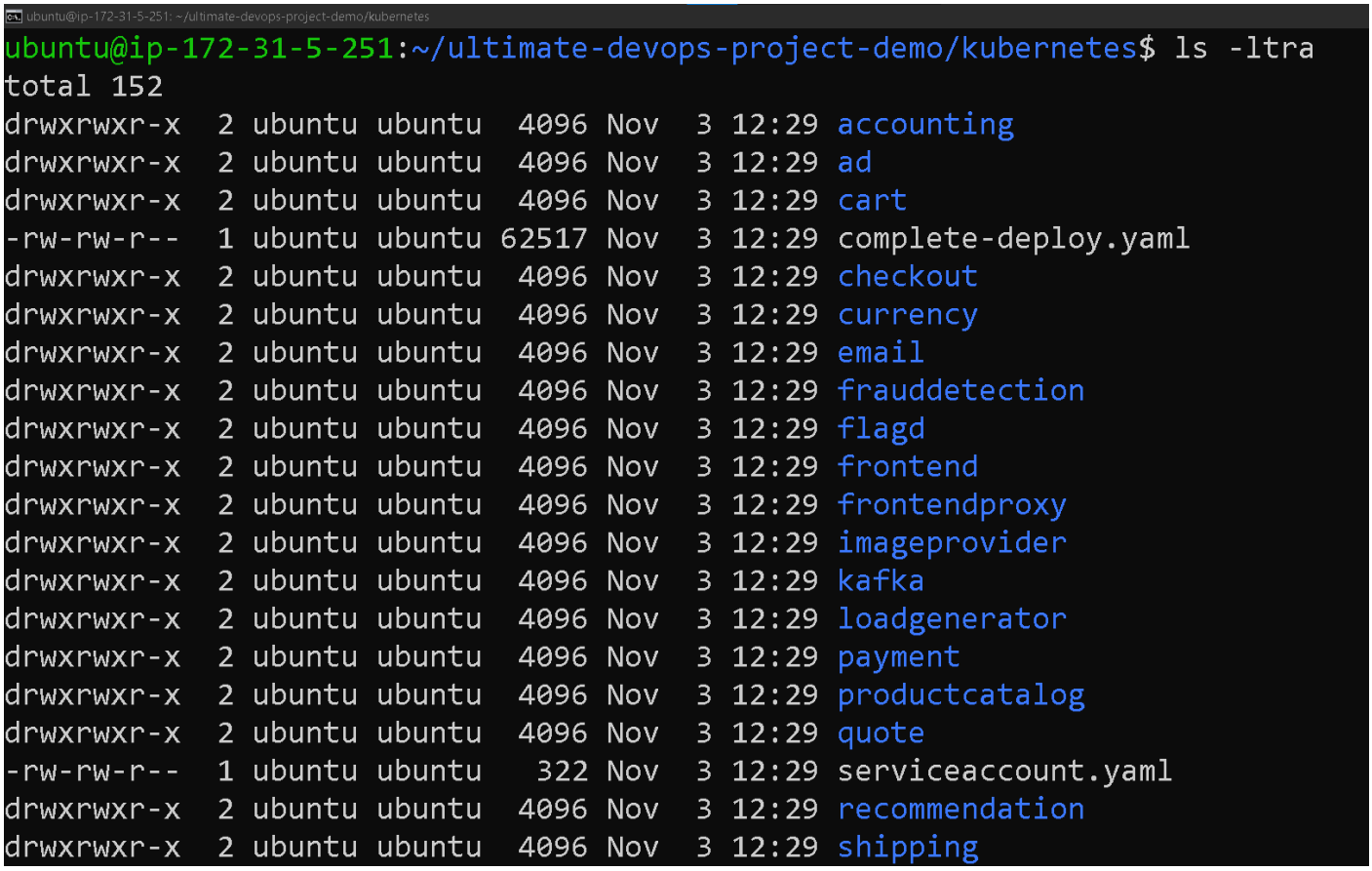
- Now our kubectl is connected with cluster.
- First we need to clone git repository where a demo project is available.

```
ubuntu@ip-172-31-5-251:~$ git clone https://github.com/akashcodes-web/ultimate-devops-proj
ect-demo.git
```

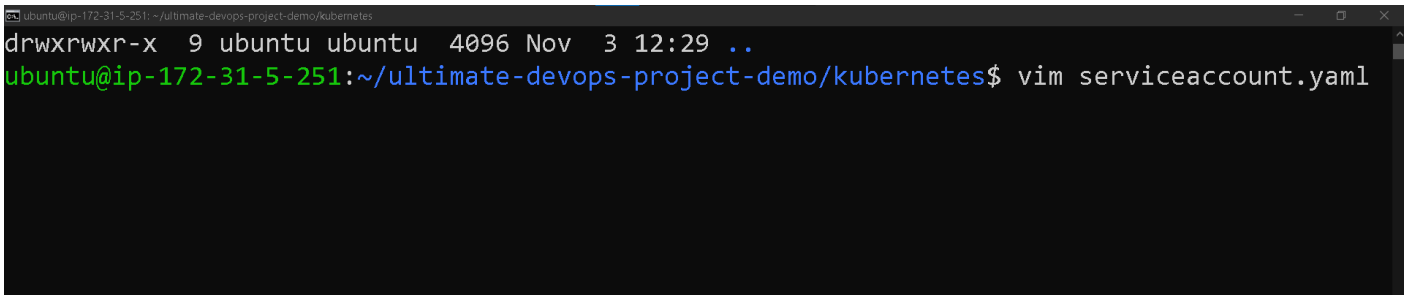
- Clone repository.

```
ubuntu@ip-172-31-5-251:~$ cd ultimate-devops-project-demo/kubernetes
```

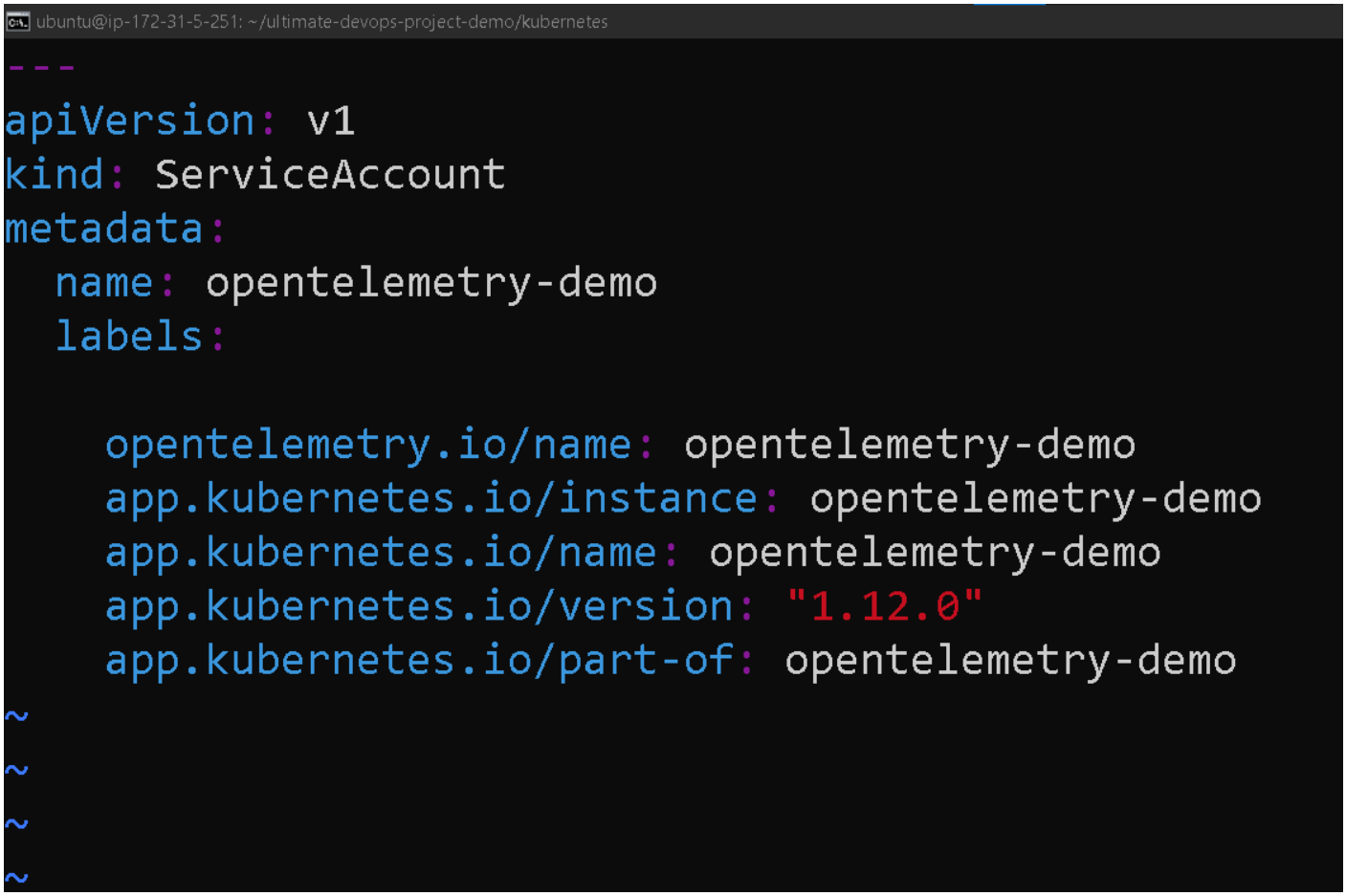
- Now go to directory.



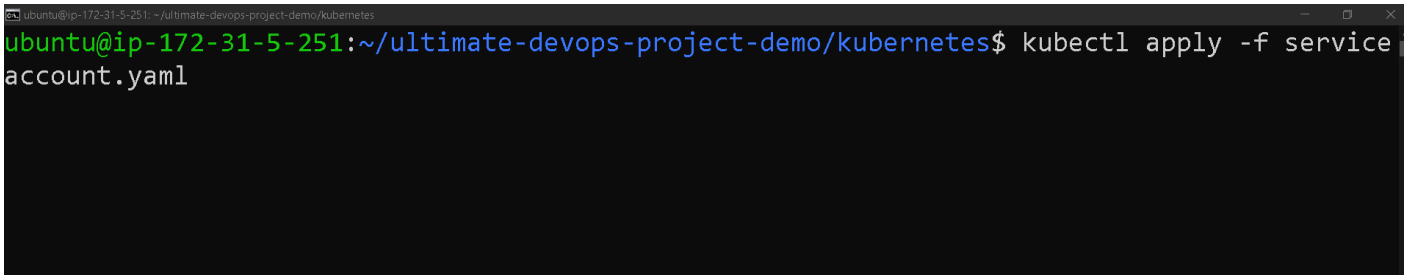
- Ls –ltra where we get our serviceaccount.yaml file.



- Now open that file.



- Now you see service account name.
- Now close it



- kubectl apply –f serviceaccount.yaml



- now check its created or not.
- Kubectl get sa

```
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$ ls
accounting            currency        frontendproxy  productcatalog       valkey
ad                    email           imageprovider  quote
cart                  flagd           kafka          recommendation
checkout              frauddetection  loadgenerator  serviceaccount.yaml
complete-deploy.yaml  frontend        payment        shipping
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$ kubectl apply complete-d
eploy.yaml
```

- Now we have there file in our project.
- Now we don't need to apply kubectl on each microservice one by one
- We have file complete-deploy.yaml where all microservice is present.
- kubectl apply –f complete-deploy.yaml

```
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$ kubectl get svc
NAME                                    TYPE        CLUSTER-IP       EXTERNAL-IP   PORT
(S)             AGE
kubernetes                              ClusterIP   172.20.0.1       <none>        443/
TCP             3h41m
opentelemetry-demo-adservice            ClusterIP   172.20.6.130     <none>        8080
/TCP            80s
opentelemetry-demo-cartservice          ClusterIP   172.20.100.244   <none>        8080
/TCP            80s
opentelemetry-demo-checkoutservice      ClusterIP   172.20.224.30    <none>        8080
/TCP            80s
opentelemetry-demo-currencyservice      ClusterIP   172.20.111.200   <none>        8080
/TCP            80s
opentelemetry-demo-emailservice         ClusterIP   172.20.143.113   <none>        8080
/TCP            80s
opentelemetry-demo-flagd                ClusterIP   172.20.36.109    <none>        8013
/TCP,4000/TCP   80s
opentelemetry-demo-frontend             ClusterIP   172.20.221.104   <none>        8080
/TCP            80s
opentelemetry-demo-frontendproxy        ClusterIP   172.20.33.206    <none>        8080
/TCP            80s
```

- kubectl get svc
- now you see our all micro services is added and deploy

```
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$ kubectl get pods
NAME                                                READY   STATUS    RESTARTS   A
GE
opentelemetry-demo-accountingservice-559bc4fd75-g52pw       1/1     Running   0          3
m5s
opentelemetry-demo-adservice-5df4bf56d6-brrjp               1/1     Running   0          3
m5s
opentelemetry-demo-cartservice-694df79ff7-lfv5p             1/1     Running   0          3
m5s
opentelemetry-demo-checkoutservice-744b6cdd6-f5ctd          1/1     Running   0          3
m5s
opentelemetry-demo-currencyservice-6b8fcd9bd6-zch4v         1/1     Running   0          3
m5s
opentelemetry-demo-emailservice-66d8476fdc-65bhr            1/1     Running   0          3
m5s
opentelemetry-demo-flagd-866b46496f-7mqqb                   2/2     Running   0          3
m5s
opentelemetry-demo-frauddetectionservice-85d48f855f-qnlgh   1/1     Running   0          3
m5s
opentelemetry-demo-frontend-59bccd8fdb-nlfhm                1/1     Running   0          3
m5s
opentelemetry-demo-frontendproxy-f46cf7d-z2ddw              1/1     Running   0          3
```

- kubectl get pods
- to see our all microservices in running state.
- Now how will you access the project.
- I need to access frontend

```
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$ kubectl get svc | grep f
rontendproxy
opentelemetry-demo-frontendproxy          ClusterIP   172.20.33.206    <none>        8080
/TCP            32m
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$
```

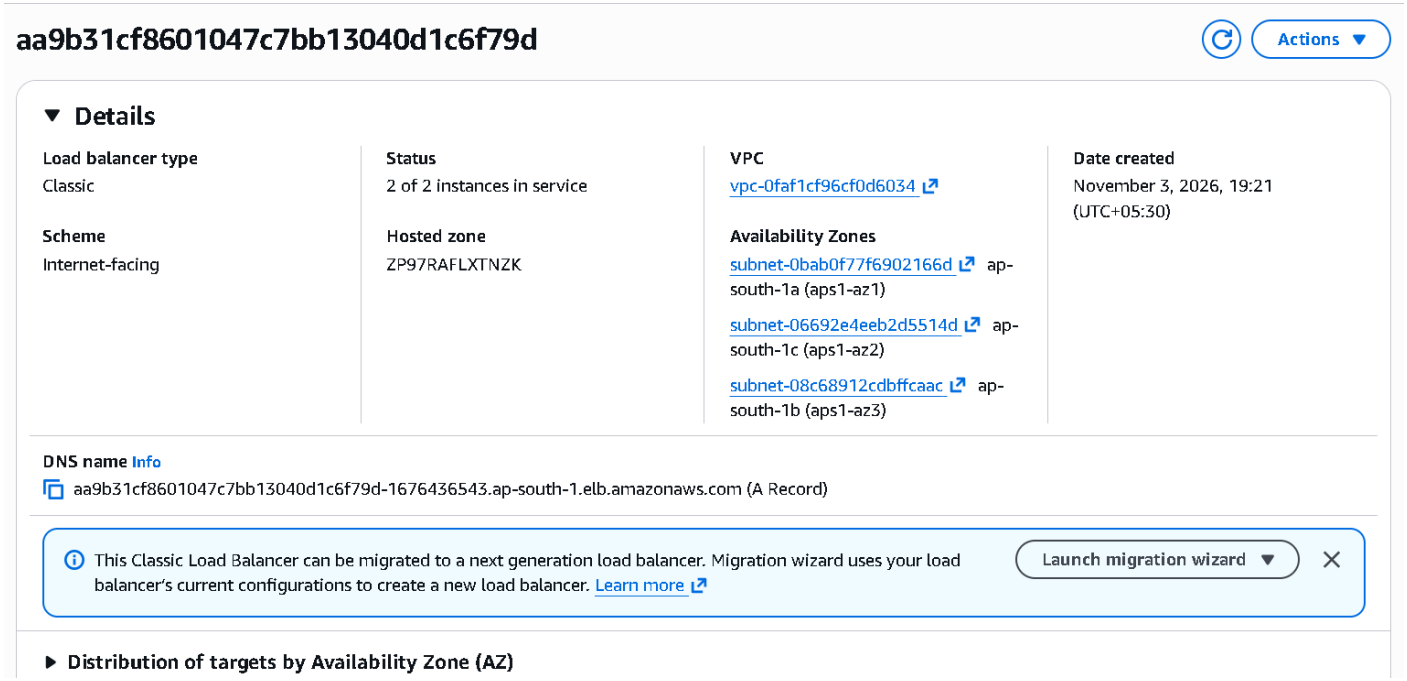
- Kubectl get svc | grep frontendproxy
- If i go to browser and try to access I cant acces it.
- Because I am not connected with eks cluster.
- Because eks cluster is in VPC

```
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$ kubectl get svc | grep f
rontendproxy
opentelemetry-demo-frontendproxy          ClusterIP   172.20.33.206    <none>        8080
/TCP            51m
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$ kubectl edit svc opentel
emetry-demo-frontendproxy
```

- kubectl edit svc opentelemetry-demo-frontendproxy

```
ubuntu@ip-172-31-5-251: ~/ultimate-devops-project-demo/kubernetes
  resourceVersion: "36739"
  uid: a9b31cf8-6010-47c7-bb13-040d1c6f79d6
spec:
  clusterIP: 172.20.33.206
  clusterIPs:
  - 172.20.33.206
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - name: tcp-service
    port: 8080
    protocol: TCP
    targetPort: 8080
  selector:
    opentelemetry.io/name: opentelemetry-demo-frontendproxy
  sessionAffinity: None
  type: LoadBalancer
status:
  loadBalancer: {}
-- INSERT --
```

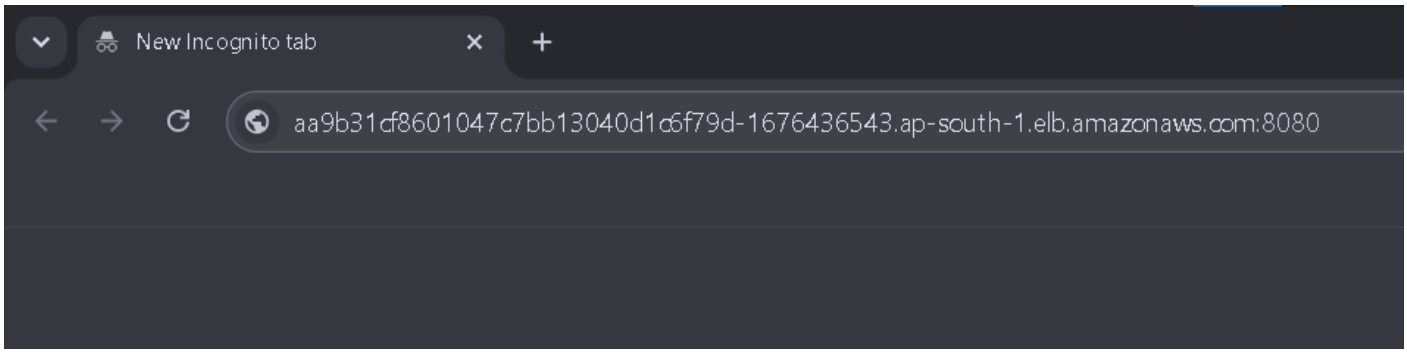
- Now simply scroll down and press i
- Then replace ClusterID to LoadBalancer
- Then press esc
- And then type :wq and press Enter
- And now wait for 5 minutes atleast,
- Because it creating Load Balancer.
- Now go to aws console and check



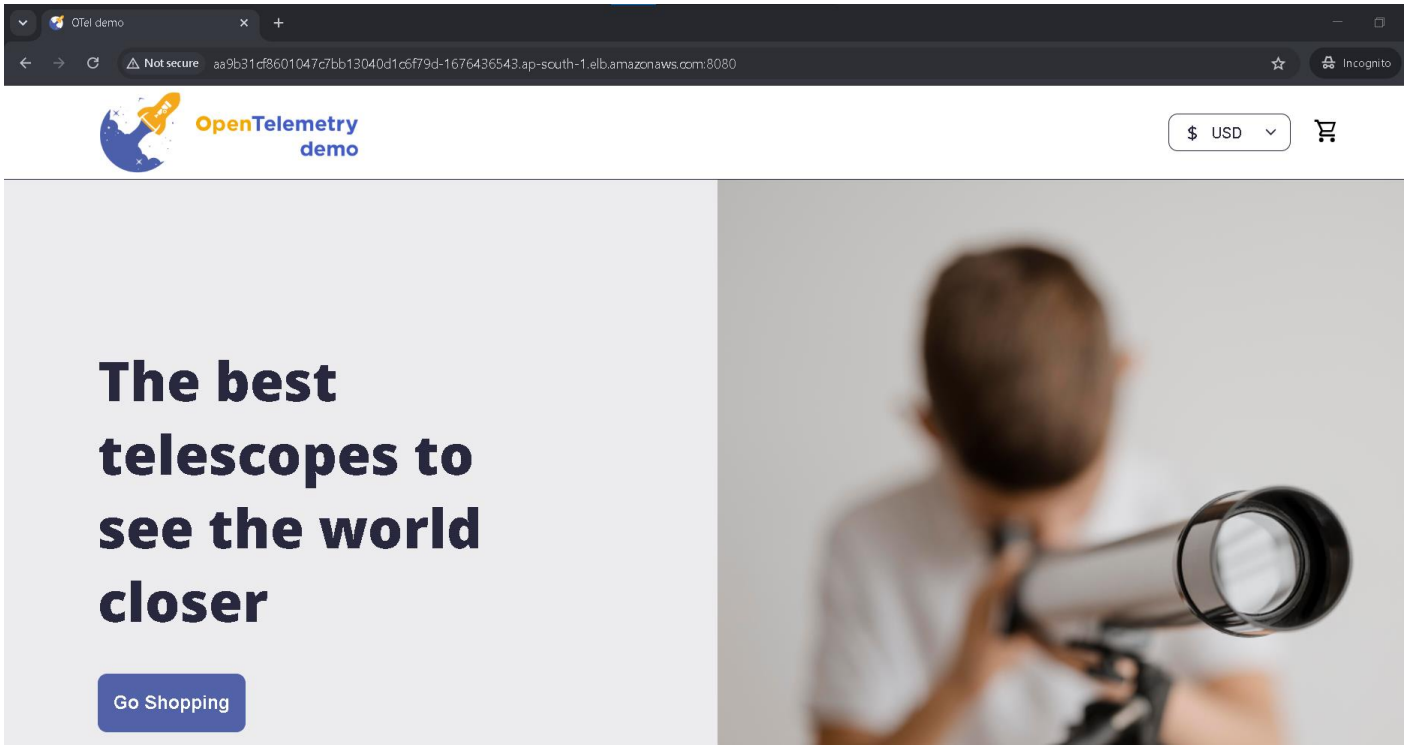
- Now see our Load Balancer is created.

```
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$ kubectl get svc | grep f
rontendproxy
opentelemetry-demo-frontendproxy          ClusterIP   172.20.33.206    <none>        8080
/TCP            51m
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$ kubectl edit svc opentel
emetry-demo-frontendproxy
service/opentelemetry-demo-frontendproxy edited
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$ kubectl get svc opentele
metry-demo-frontendproxy
NAME                                TYPE           CLUSTER-IP      EXTERNAL-IP
                                                    PORT(S)          AGE
opentelemetry-demo-frontendproxy    LoadBalancer   172.20.33.206   aa9b31cf8601047c7bb13040
d1c6f79d-1676436543.ap-south-1.elb.amazonaws.com    8080:32559/TCP   62m
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes$
```

- Now see we get external IP.
- Kubectl get svc opetelementry-demo-frontendproxy
- Now copy this and go to the browser tab



- Paste in search bar and add :8080



- Now see our site is accessible.
- Now we deploy ALB Ingress controller

- Because if we want to change our URL we change it With AWS Console(UI)
- But user don't know how this will happened so.
- Now go to command prompt.

```
ubuntu@ip-172-31-5-251:~$ export cluster_name=my-eks-cluster
ubuntu@ip-172-31-5-251:~$ oidc_id=$(aws eks describe-cluster --name $cluster_name --query
"cluster.identity.oidc.issuer" --output text | cut -d '/' -f 5)
```

- export cluster_name=my-eks-cluster
- oidc_id=$(aws eks describe-cluster --name $cluster_name --query "cluster.identity.oidc.issuer" --output text | cut -d '/' -f 5)

```
ubuntu@ip-172-31-5-251:~$ export cluster_name=my-eks-cluster
ubuntu@ip-172-31-5-251:~$ oidc_id=$(aws eks describe-cluster --name $cluster_name --query
"cluster.identity.oidc.issuer" --output text | cut -d '/' -f 5)
ubuntu@ip-172-31-5-251:~$ echo $oidc_id
A271FB8EE919A407B8A67813B1C1F6C6
ubuntu@ip-172-31-5-251:~$
```

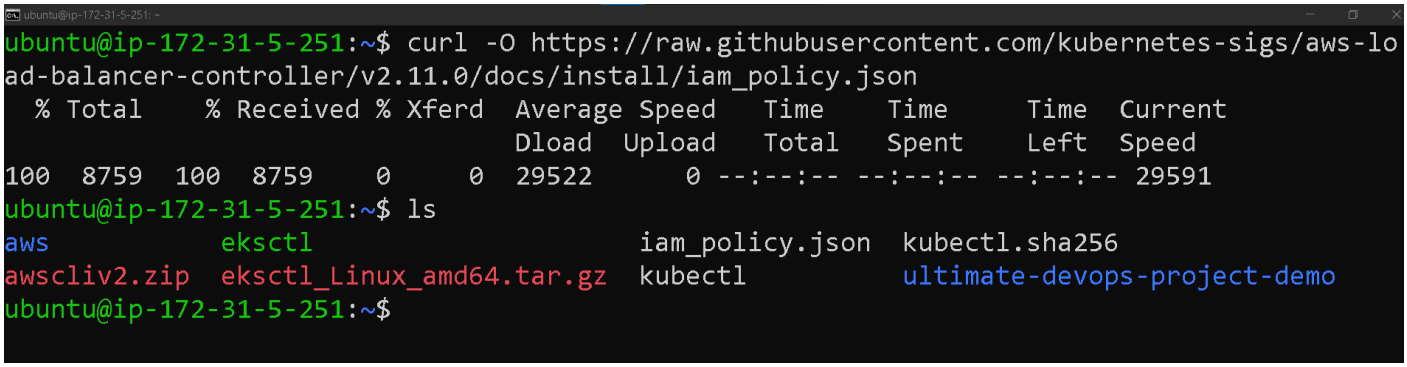
- echo $oidc_id

```
ubuntu@ip-172-31-5-251:~$ ARCH=amd64
ubuntu@ip-172-31-5-251:~$ PLATFORM=$(uname -s)_$ARCH
ubuntu@ip-172-31-5-251:~$ curl -sLO "https://github.com/eksctl-io/eksctl/releases/latest/d
ownload/eksctl_${PLATFORM}.tar.gz"
ubuntu@ip-172-31-5-251:~$ tar -xzf eksctl_${PLATFORM}.tar.gz
ubuntu@ip-172-31-5-251:~$ sudo install -m 0755 eksctl /usr/local/bin/eksctl
ubuntu@ip-172-31-5-251:~$ eksctl version
0.231.0
ubuntu@ip-172-31-5-251:~$ eksctl utils associate-iam-oidc-provider --cluster $cluster_name
 --approve
2026-11-03 14:37:22 [ℹ]  will create IAM Open ID Connect provider for cluster "my-eks-clus
ter" in "ap-south-1"
2026-11-03 14:37:23 [✔]  created IAM Open ID Connect provider for cluster "my-eks-cluster"
 in "ap-south-1"
ubuntu@ip-172-31-5-251:~$
```

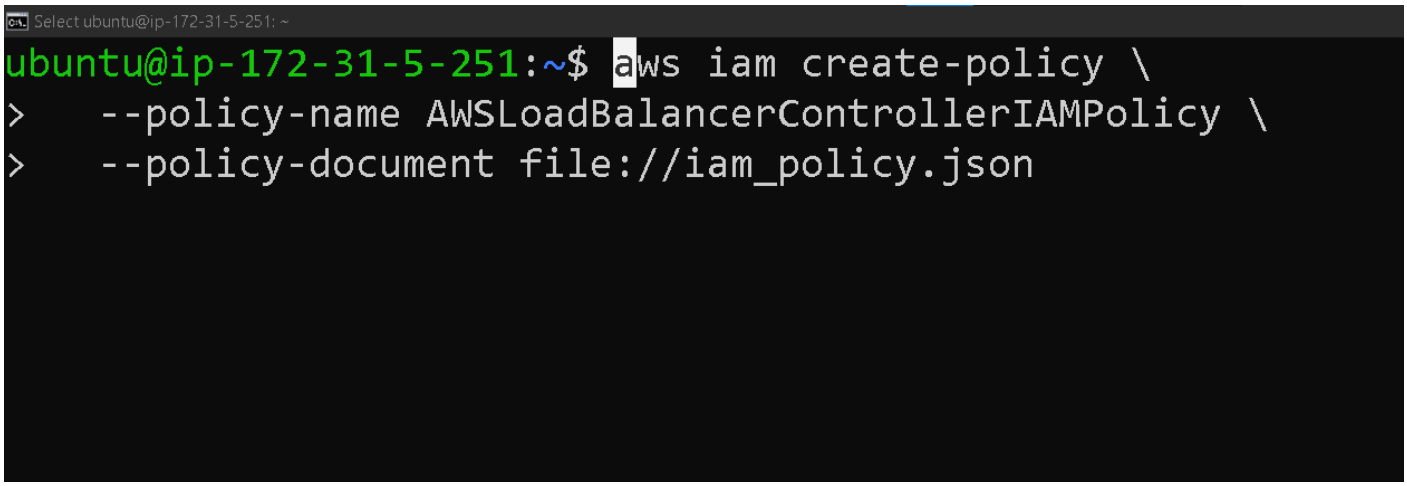
- eksctl utils associate-iam-oidc-provider --cluster $cluster_name –approve
- so basically id added IAM oidc provider to your cluster.
- It not work firstly we need to install eksctl in our machine.
- eksctl code in resource.



- curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.11.0/docs/install/iam_policy.json
- The command downloads the IAM policy JSON file for AWS Load Balancer Controller version v2.11.0



- Now check its added or not.



- command to create an IAM policy for the AWS Load Balancer Controller.
- These commands in resource.

```
ubuntu@ip-172-31-5-251:~$ eksctl create iamserviceaccount \
>   --cluster=my-eks-cluster \
>   --region=ap-south-1 \
>   --namespace=kube-system \
>   --name=aws-load-balancer-controller \
>   --role-name=AmazonEKSLoadBalancerControllerRole \
>   --attach-policy-arn=arn:aws:iam::463646775279:policy/AWSLoadBalancerControllerIAMPolic
y \
>   --override-existing-serviceaccounts \
>   --approve
2026-11-03 15:37:53 [ℹ]  1 existing iamserviceaccount(s) (kube-system/aws-load-balancer-co
ntroller) will be excluded
2026-11-03 15:37:53 [ℹ]  1 iamserviceaccount (kube-system/aws-load-balancer-controller) wa
s excluded (based on the include/exclude rules)
2026-11-03 15:37:53 [!]  metadata of serviceaccounts that exist in Kubernetes will be upda
ted, as --override-existing-serviceaccounts was set
2026-11-03 15:37:53 [ℹ]  no tasks
ubuntu@ip-172-31-5-251:~$
```

- This is provided in resource.
- Now we install helm

```
ubuntu@ip-172-31-5-251:~$ helm version
version.BuildInfo{Version:"v3.22.1", GitCommit:"d6c1a0e4027caa6abc7892e7ea9f0cda7447baa3",
 GitTreeState:"clean", GoVersion:"go1.26.9"}
ubuntu@ip-172-31-5-251:~$
```

- Installation code is provided in resource.

```
ubuntu@ip-172-31-5-251:~$ helm repo add eks https://aws.github.io/eks-charts
"eks" has been added to your repositories
ubuntu@ip-172-31-5-251:~$ helm repo update eks
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "eks" chart repository
Update Complete. ⎈Happy Helming!⎈
ubuntu@ip-172-31-5-251:~$ helm install aws-load-balancer-controller eks/aws-load-balancer-
controller -n kube-system --set clusterName=my-eks-cluster --set serviceAccount.create=fal
se --set serviceAccount.name=aws-load-balancer-controller --set region=ap-south-1 --set vp
cId=vpc-0faf1cf96cf0d6034

NAME: aws-load-balancer-controller
LAST DEPLOYED: Tue Nov  3 15:54:43 2026
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
AWS Load Balancer controller installed!
ubuntu@ip-172-31-5-251:~$
ubuntu@ip-172-31-5-251:~$
```

- These code are provided in resource
- Now check Alb ports are up and running

```
ubuntu@ip-172-31-5-251:~$ kubectl get pods -n kube-system
NAME                                           READY   STATUS    RESTARTS   AGE
aws-load-balancer-controller-85cf68845f-2pj2f  1/1     Running   0          3m21s
aws-load-balancer-controller-85cf68845f-9jnnb  1/1     Running   0          3m21s
aws-node-6v72w                                 2/2     Running   0          6h13m
aws-node-qtwmn                                 2/2     Running   0          6h12m
coredns-6f75fb579c-gqkkm                       1/1     Running   0          6h55m
coredns-6f75fb579c-p7tk4                       1/1     Running   0          6h55m
kube-proxy-kr6g8                               1/1     Running   0          6h13m
kube-proxy-rhrms                               1/1     Running   0          6h12m
ubuntu@ip-172-31-5-251:~$
```

- kubectl get pods –n kube-system
- if these two item not seen then run some code that provided in resource

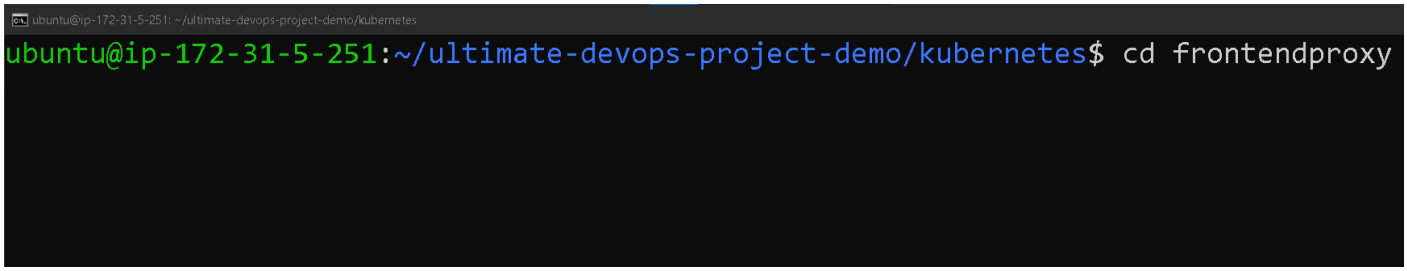
```
ubuntu@ip-172-31-5-251:~$ kubectl edit svc opentelemetry-demo-frontendproxy
```

kubectl edit svc opentelemetry-demo-frontendproxy

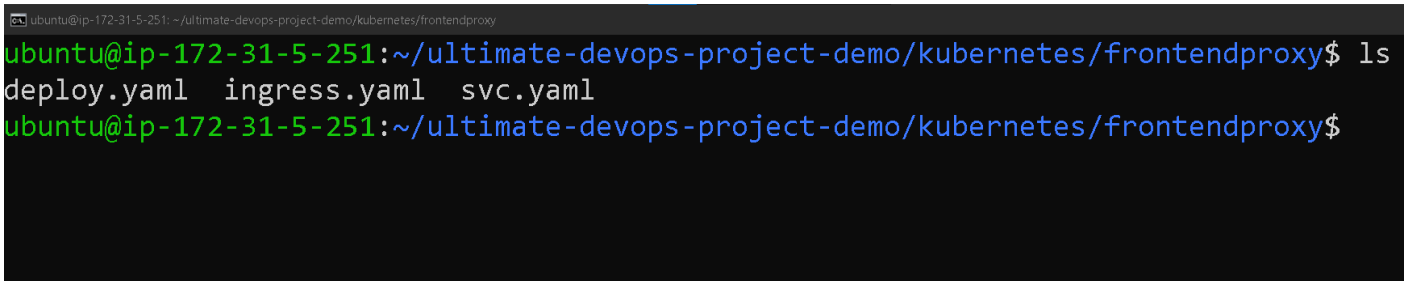
- open it

```
  clusterIP: 172.20.33.206
  clusterIPs:
  - 172.20.33.206
  externalTrafficPolicy: Cluster
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - name: tcp-service
    nodePort: 32559
    port: 8080
    protocol: TCP
    targetPort: 8080
  selector:
    opentelemetry.io/name: opentelemetry-demo-frontendproxy
  sessionAffinity: None
  type: NodePort
status:
  loadBalancer: {}
```

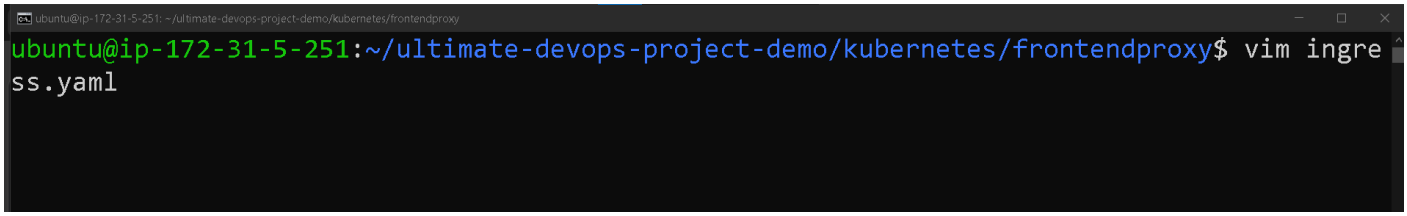
- now scroll down and change the type from LoadBalancer to NodePort.
- The press Esc and then type :wq and press Enter



- Go to directory cd ultimate-devops-project-demo/kubernetes
- Then cd frontendproxy



- In this these files are present.



- Lets create this vim ingress.yaml

```
ubuntu@ip-172-31-5-251: ~/ultimate-devops-project-demo/kubernetes/frontendproxy
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend-proxy
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
spec:
  ingressClassName: alb
  rules:
    - host: example.com
      http:
        paths:
          - path: "/"
            pathType: Prefix
            backend:
              service:
                name: opentelemetry-demo-frontendproxy
                port:
                  number: 8080
"ingress.yaml" 20L, 488B
```

- This file is present in this file we have api version meta data name and host.
- Basically we have not a host like example.com it's a dummy host.
- In real world like example we buy host from godaddy and the host I provide here.
- Simply quit by pressing :q!

```
ubuntu@ip-172-31-5-251: ~/ultimate-devops-project-demo/kubernetes/frontendproxy
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes/frontendproxy$ kubectl ap
ply -f ingress.yaml
```

kubectl apply -f ingress.yaml

- So that our ingress controller read this

policy - Notepad

File Edit Format View Help

```
-----------------Create policy----------------------
cat > trust-policy.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::463646775279:oidc-provider/oidc.eks.ap-south-1.amazonaws.com/id/A271FB8EE919A407B8A67813B1C1F6C6"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "oidc.eks.ap-south-1.amazonaws.com/id/A271FB8EE919A407B8A67813B1C1F6C6:aud": "sts.amazonaws.com",
          "oidc.eks.ap-south-1.amazonaws.com/id/A271FB8EE919A407B8A67813B1C1F6C6:sub": "system:serviceaccount:kube-system:aws-load-balancer-controller"
        }
      }
    }
  ]
}
EOF


------------------update IAM Role -------------------------------

aws iam update-assume-role-policy \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --policy-document file://trust-policy.json


---------------Attach Policy -------------------------

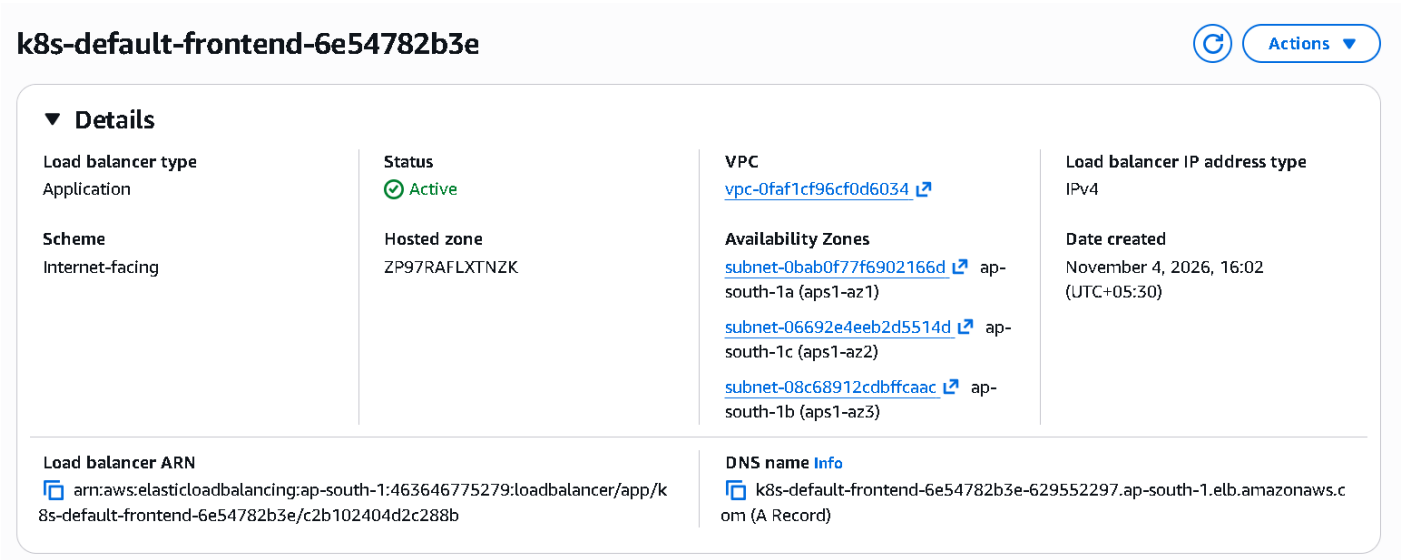
aws iam attach-role-policy \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --policy-arn arn:aws:iam::463646775279:policy/AWSLoadBalancerControllerIAMPolicy
```

- Now then you need to create and attach policies.
- These policies is provided in resource.
- Simply one by one execute.

```
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes/frontendproxy$ kubectl ge
t ingress
NAME             CLASS   HOSTS         ADDRESS
                      PORTS   AGE
frontend-proxy   alb     example.com   k8s-default-frontend-6e54782b3e-629552297.ap-south-
1.elb.amazonaws.com   80      23m
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes/frontendproxy$
```

kubectl get ingress

- You get a new address and you go and also see in your aws console load balancer is created.



- Now see one Load balancer is created.
- Now copy the dns name.
- Go back to Command prompt.

```
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes/frontendproxy$ nslookup k
8s-default-frontend-6e54782b3e-629552297.ap-south-1.elb.amazonaws.com
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
Name:   k8s-default-frontend-6e54782b3e-629552297.ap-south-1.elb.amazonaws.com
Address: 13.206.67.152
Name:   k8s-default-frontend-6e54782b3e-629552297.ap-south-1.elb.amazonaws.com
Address: 3.6.82.255
Name:   k8s-default-frontend-6e54782b3e-629552297.ap-south-1.elb.amazonaws.com
Address: 13.207.70.19

ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes/frontendproxy$
```

- Now type nslookup and copied DNS
- Now you see address copy it.

```
ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes/frontendproxy$ nslookup k
8s-default-frontend-6e54782b3e-629552297.ap-south-1.elb.amazonaws.com
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
Name:   k8s-default-frontend-6e54782b3e-629552297.ap-south-1.elb.amazonaws.com
Address: 13.206.67.152
Name:   k8s-default-frontend-6e54782b3e-629552297.ap-south-1.elb.amazonaws.com
Address: 3.6.82.255
Name:   k8s-default-frontend-6e54782b3e-629552297.ap-south-1.elb.amazonaws.com
Address: 13.207.70.19

ubuntu@ip-172-31-5-251:~/ultimate-devops-project-demo/kubernetes/frontendproxy$ logout
Connection to 13.201.230.150 closed.

C:\Users\Akash Kumar\Downloads>
```

- Now go to local machine.
- click the Windows Start button and search for Notepad.
- Right-click Notepad → Run as administrator → click Yes.
- In Notepad, select File → Open.
- Go to: C:\Windows\System32\drivers\etc
- Change the file type to All Files (.).
- Open hosts.

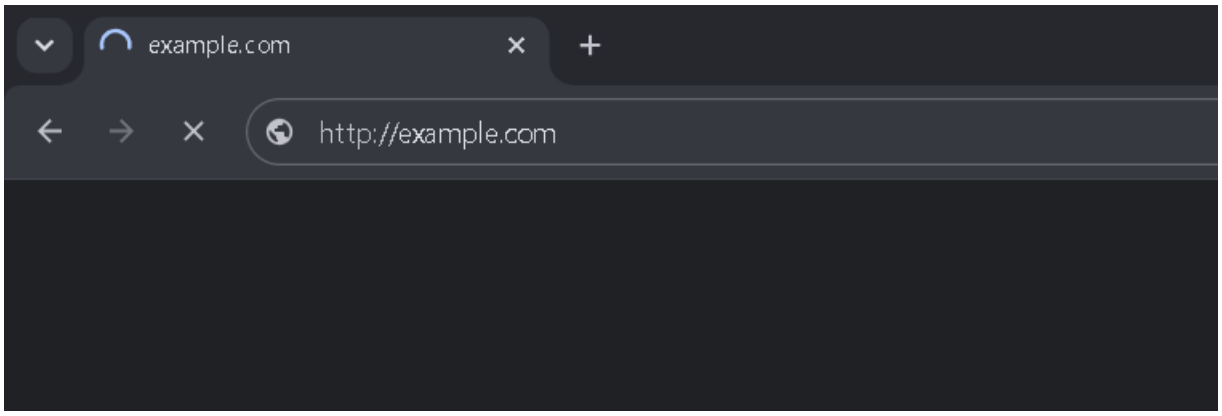
*hosts - Notepad

File Edit Format View Help

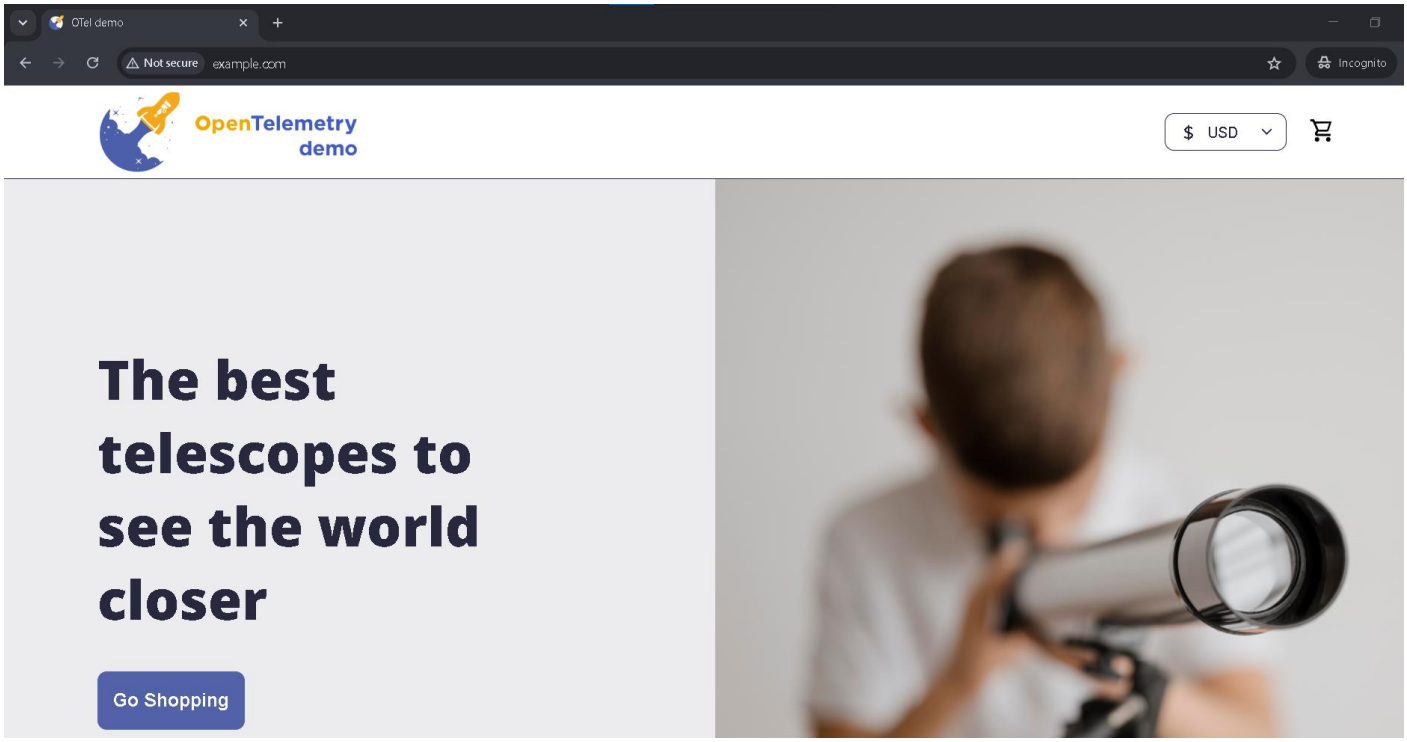
```
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#      102.54.94.97     rhino.acme.com          # source server
#       38.25.63.10     x.acme.com              # x client host

# localhost name resolution is handled within DNS itself.
#       127.0.0.1       localhost
#       ::1             localhost
#127.0.0.1 194.41.113.158
13.207.70.19   example.com
```

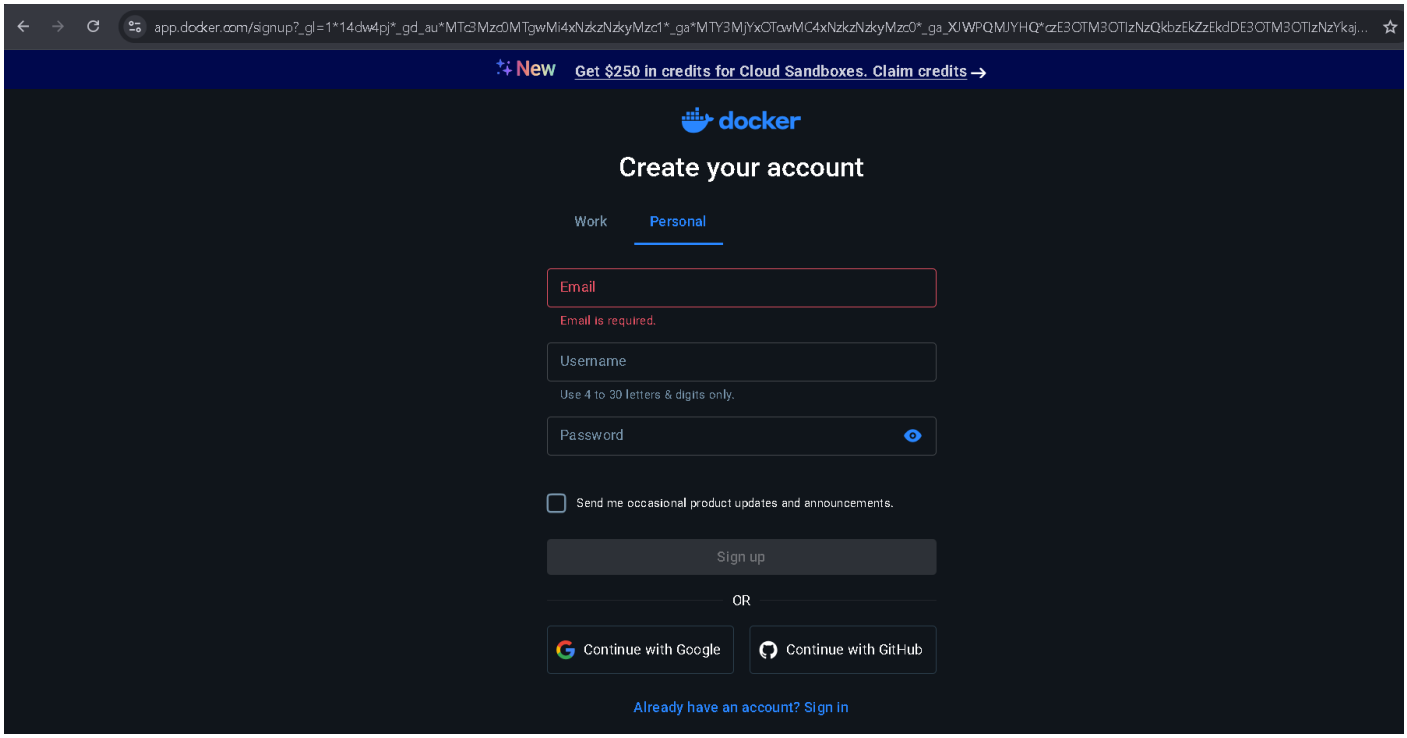
- This file is open simply add ip address and example.com
- Then save it.
- Now open browser.



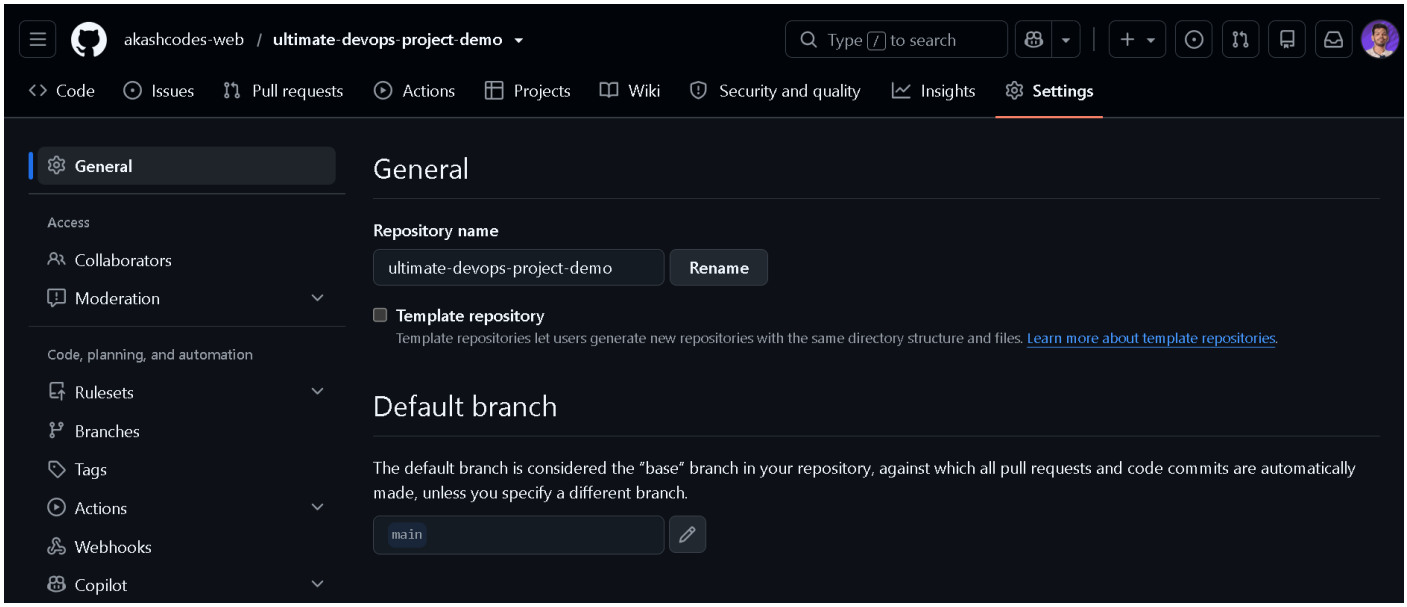
- And type this http://example.com



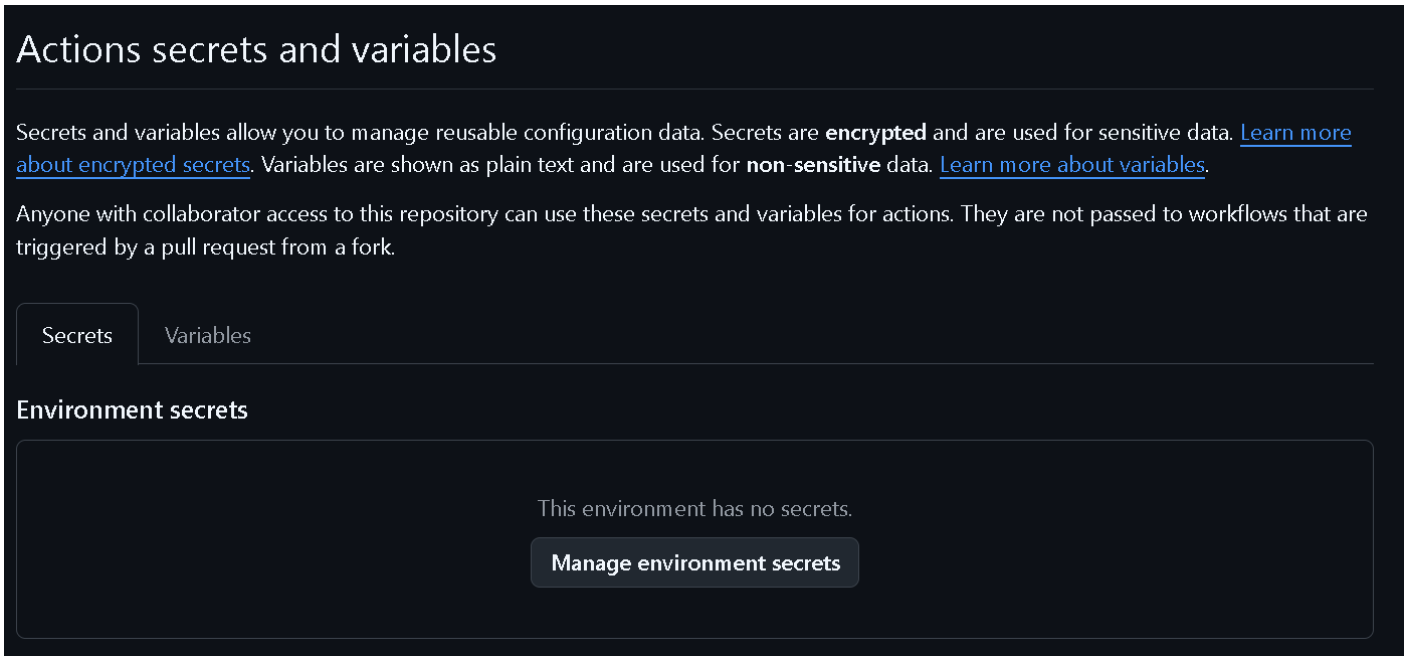
- Now see it opens.
- Now we work on our CI/CD part
- CI is continues integration and CD for Continues Delivery
- Why its use like if you pull you project in git but there is some security issues that you not check and you the the x person that pull this result in git then it also not check.
- So remove this manual effort we implement CI/CD



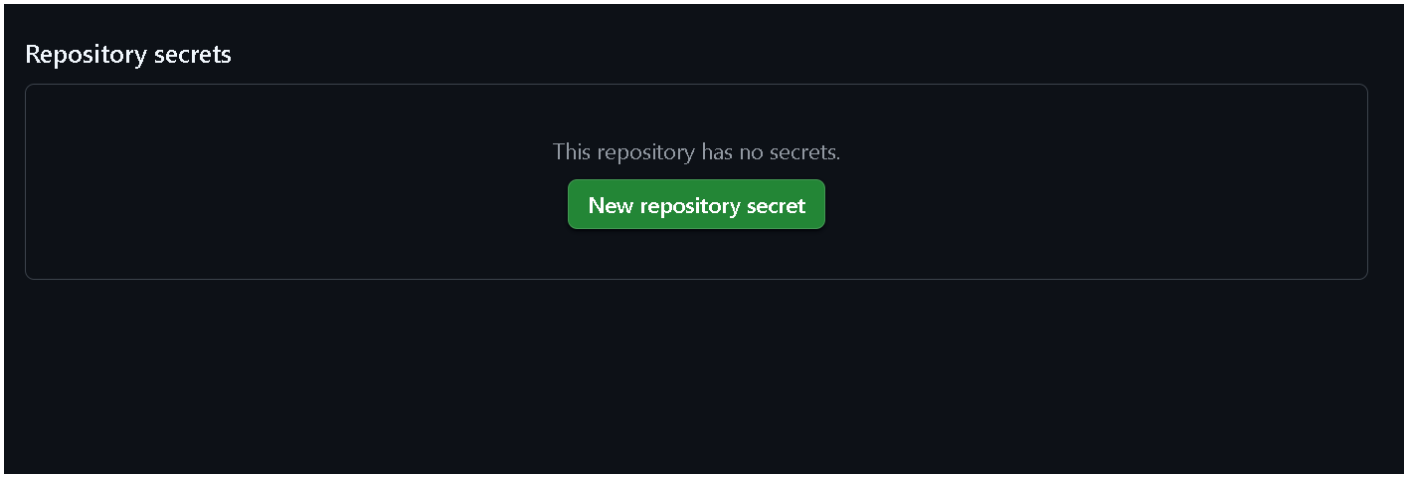
- Now lets create your docker account.
- And then go to git hub repository where your demo app is present.



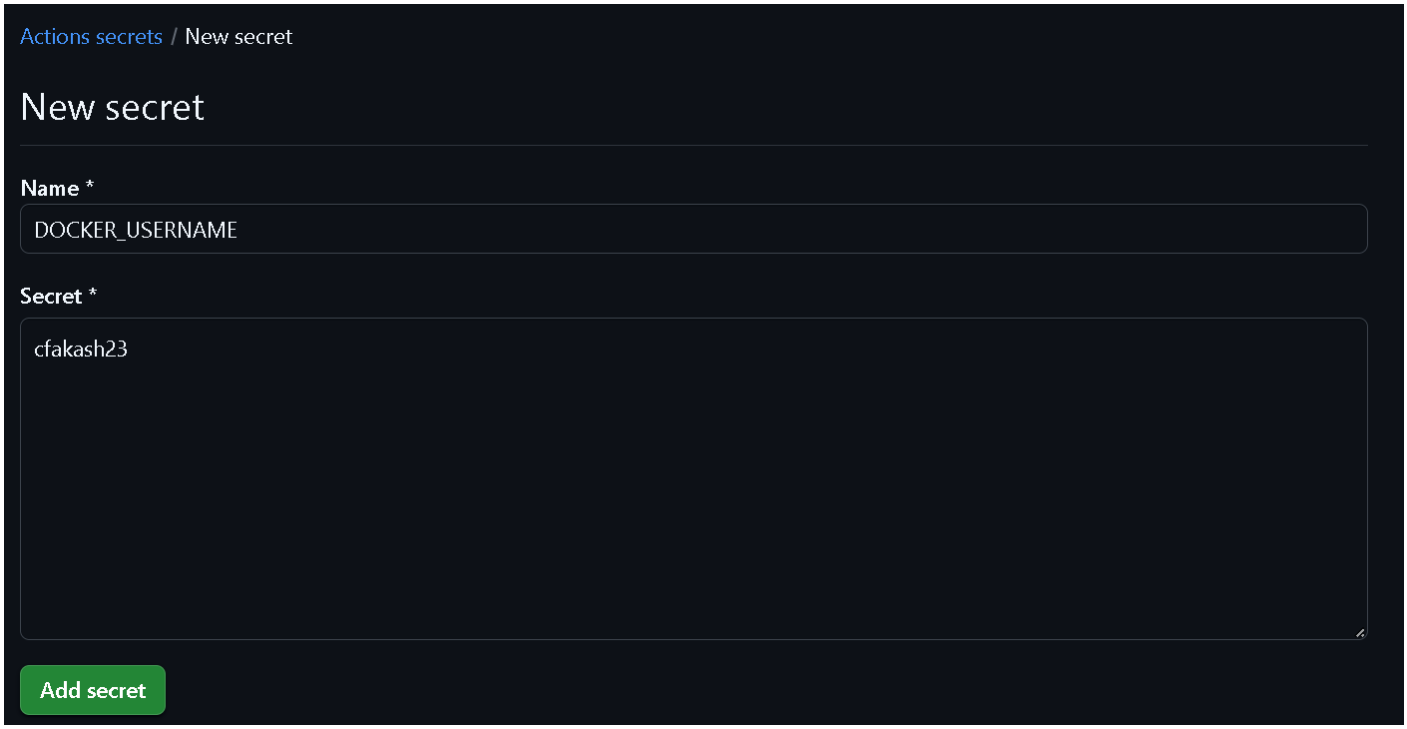
- Now go to setting
- And scroll down



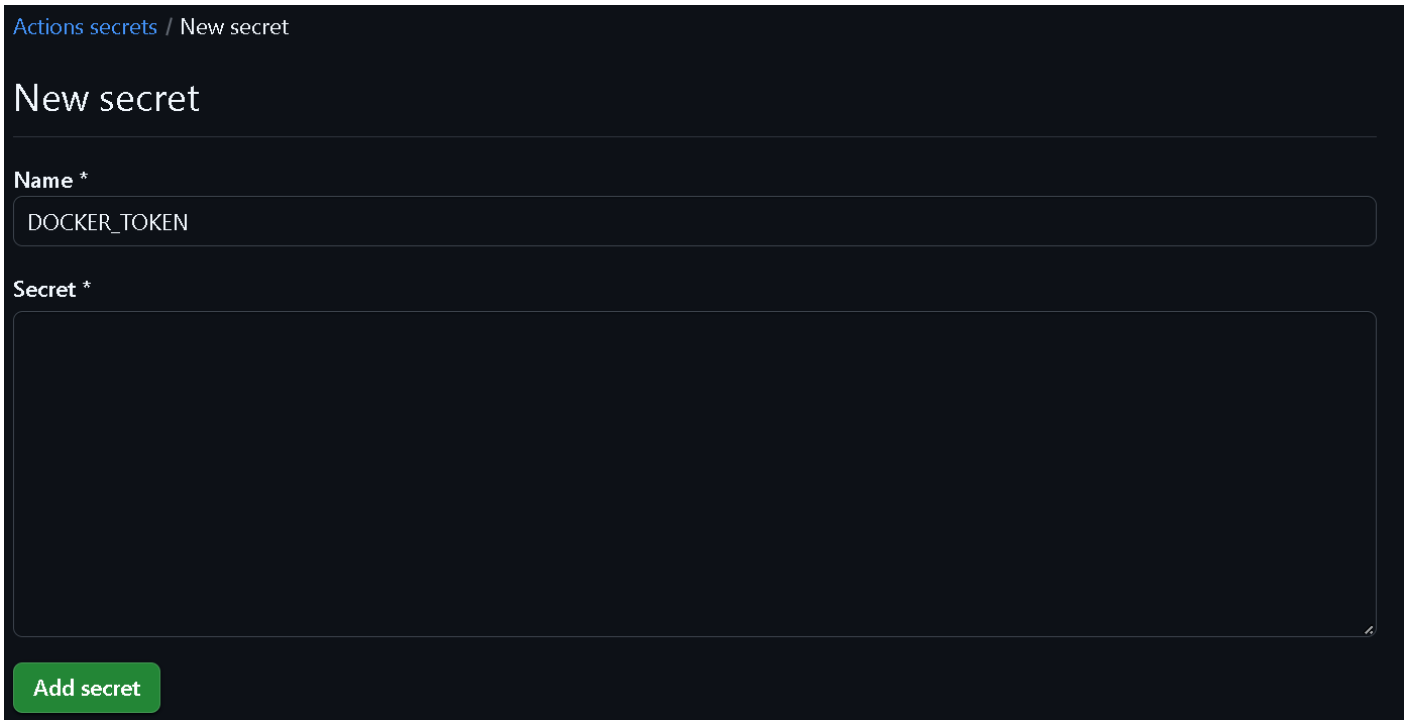
- In nav bar select secret and variables and select action.
- Then in action scroll down



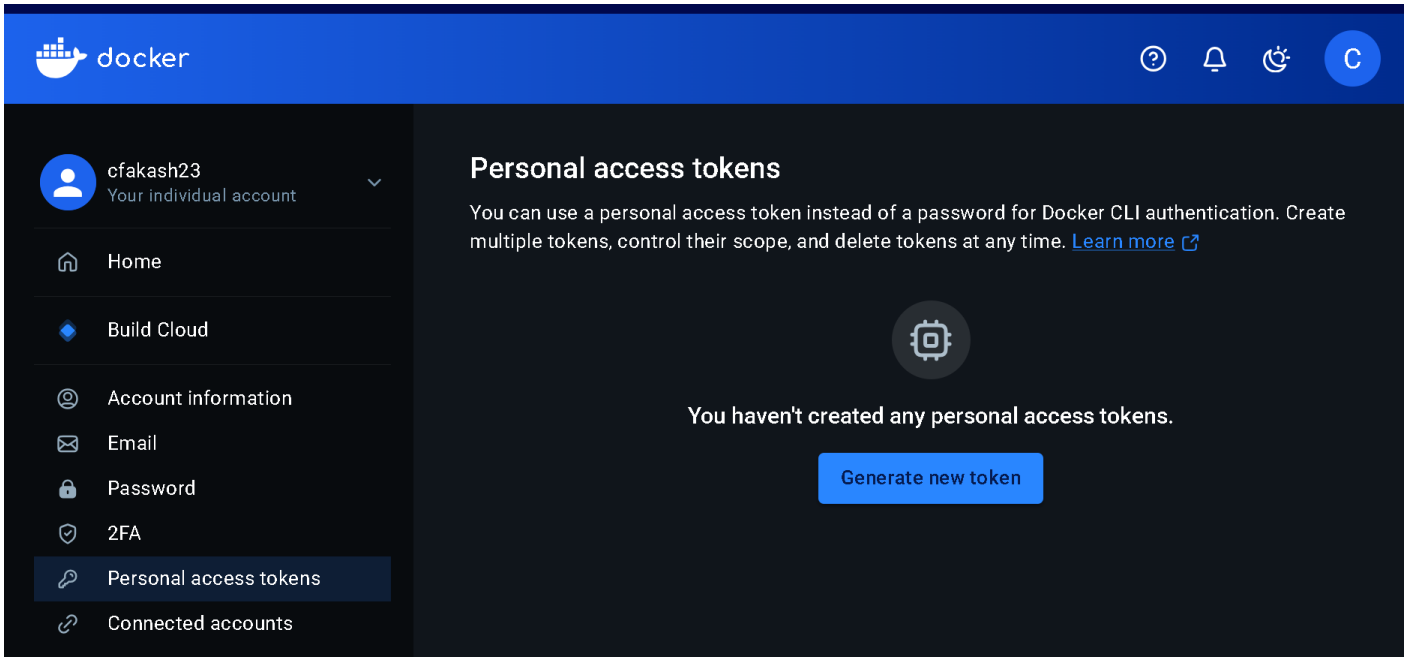
- Click New repository secret.



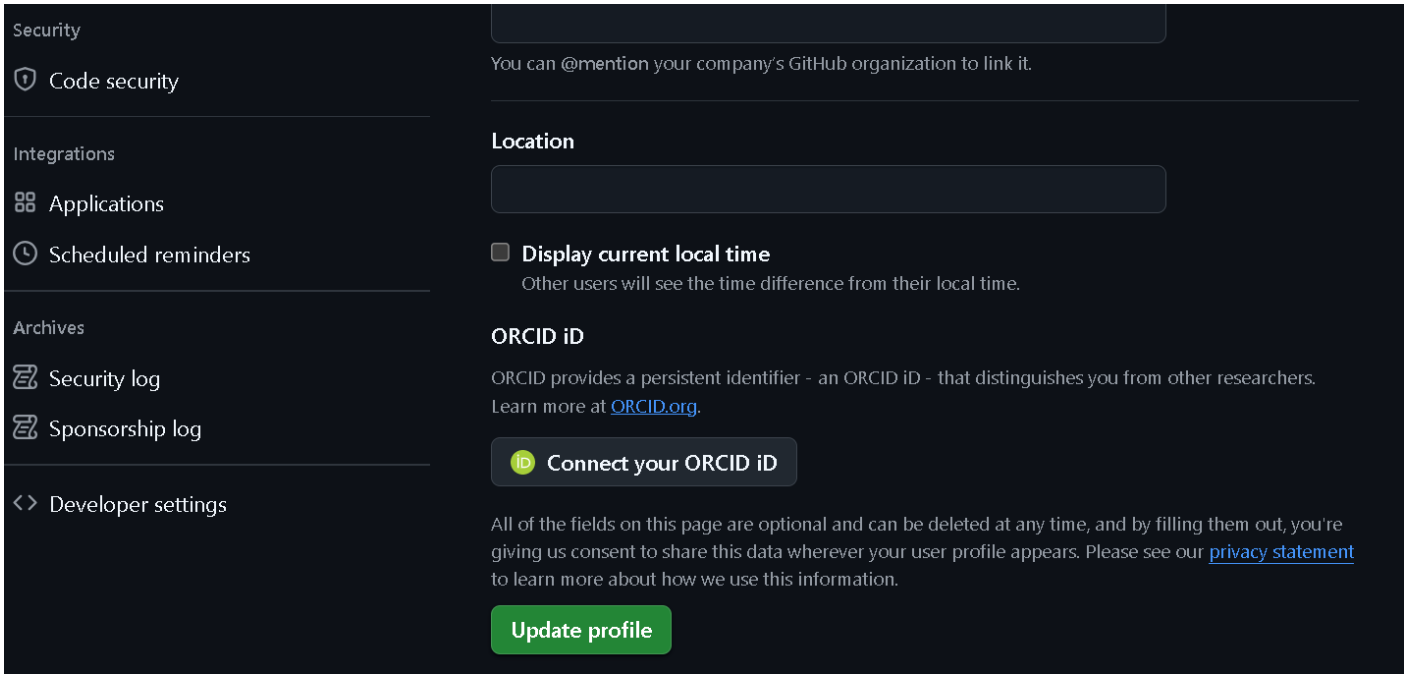
- Add this and click Add secret.



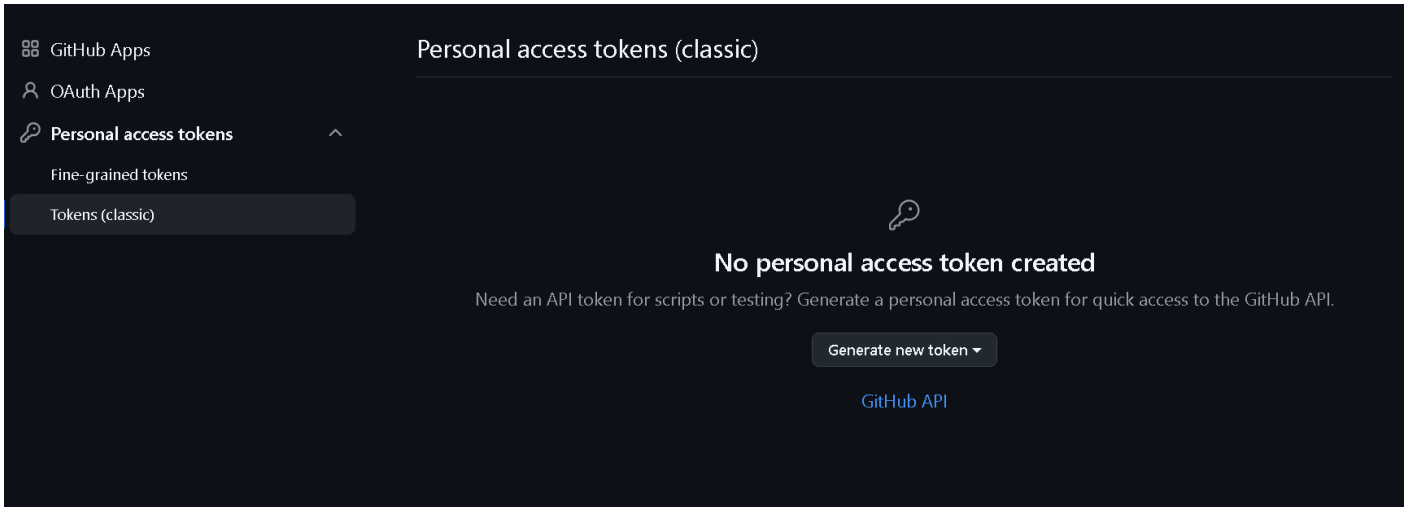
- Now we need to add one more secret DOCKER_TOKEN.
- How we get this token.
- Go to docker account.



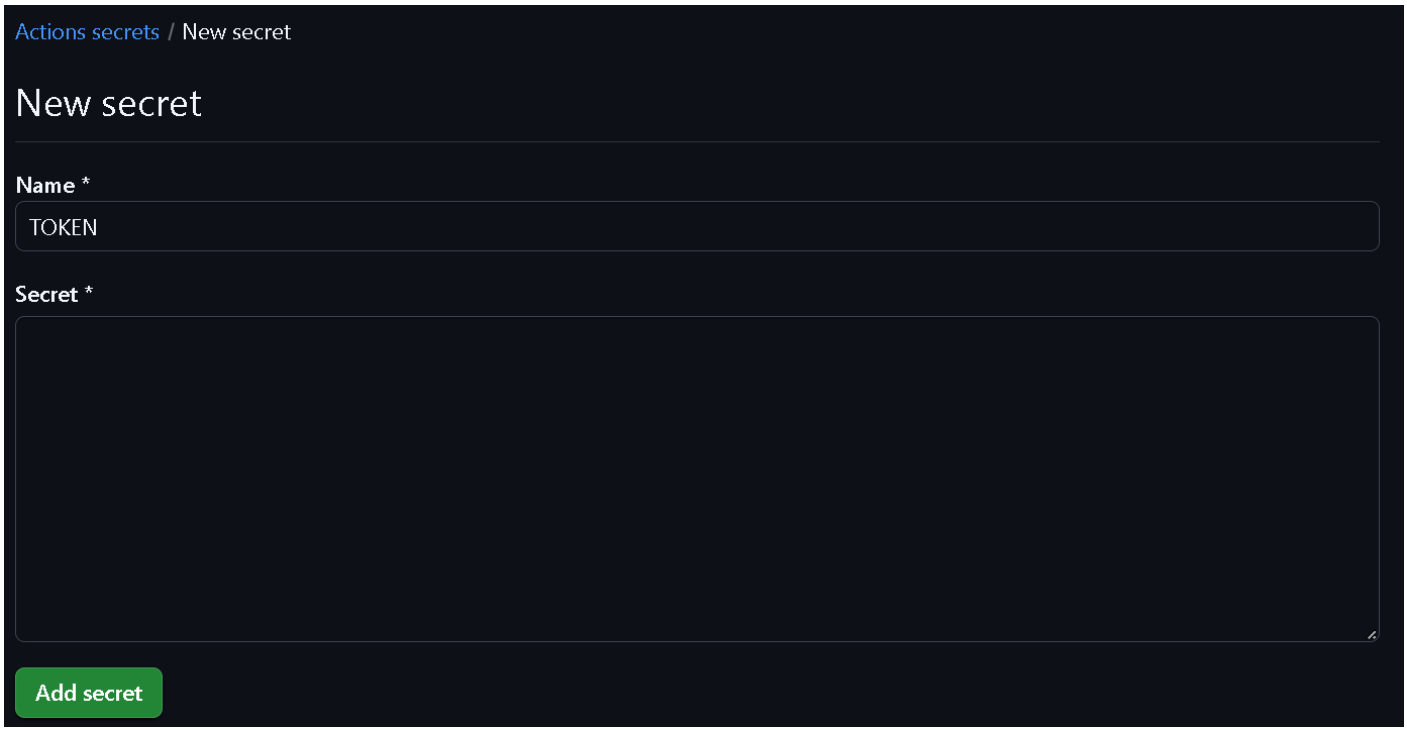
- Now go to account setting and click Personal access tokens.
- And then click generate new tokens
- And then add in github secret.
- Now you need git token.
- So go to your git hub account.



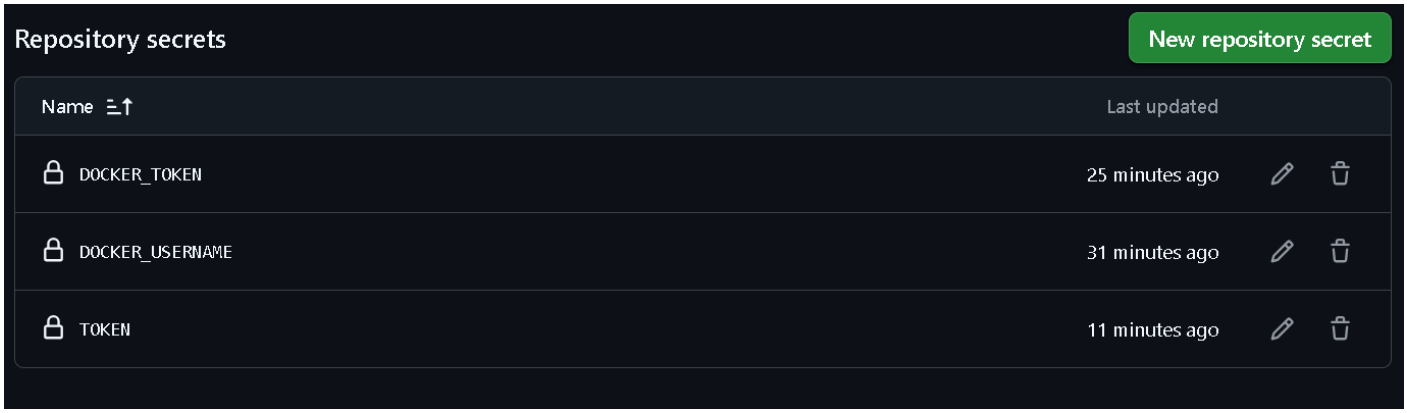
- In github go to settings and then select Developer settings



- Now in personal access token click Token classic
- And click Generate new token classic.
- Then give all permission and copy token.
- Then go to repository where demo project is present.

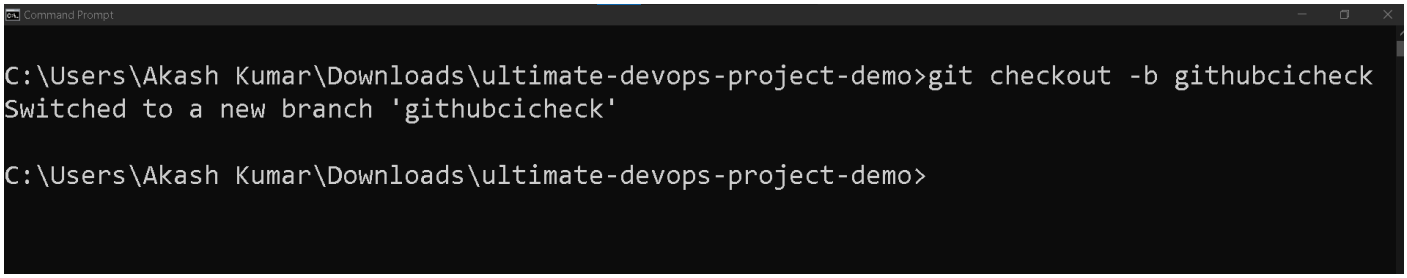


- Then again add new secret and paste token

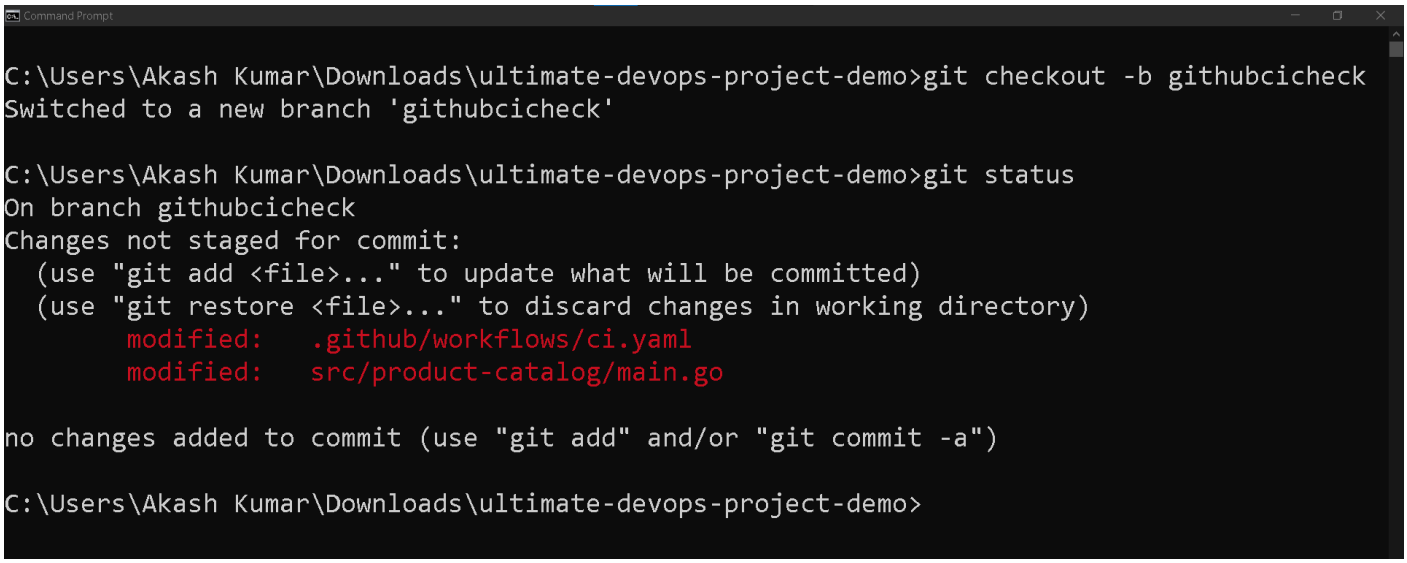


- Now we have three repository secret

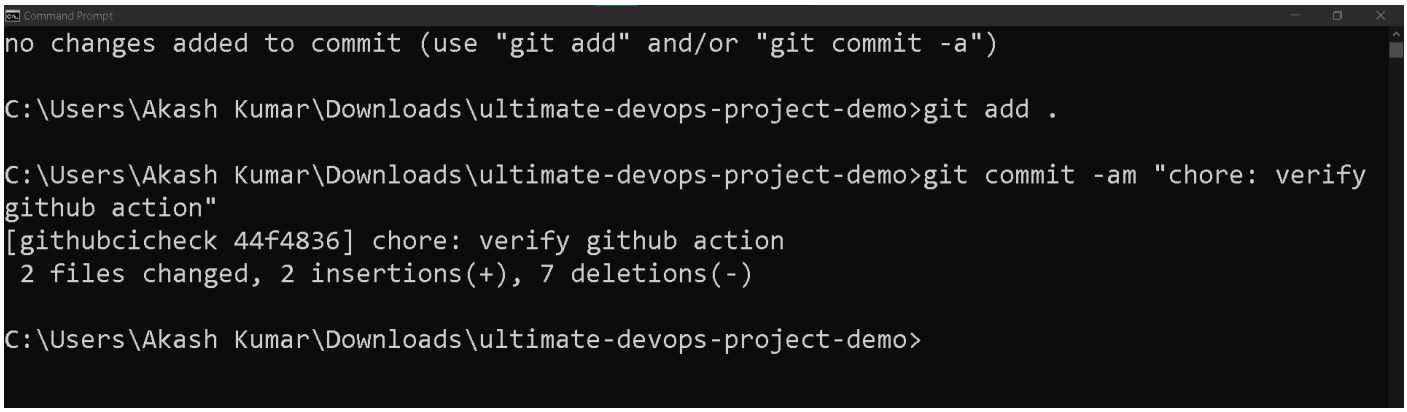
- Now go to cmd



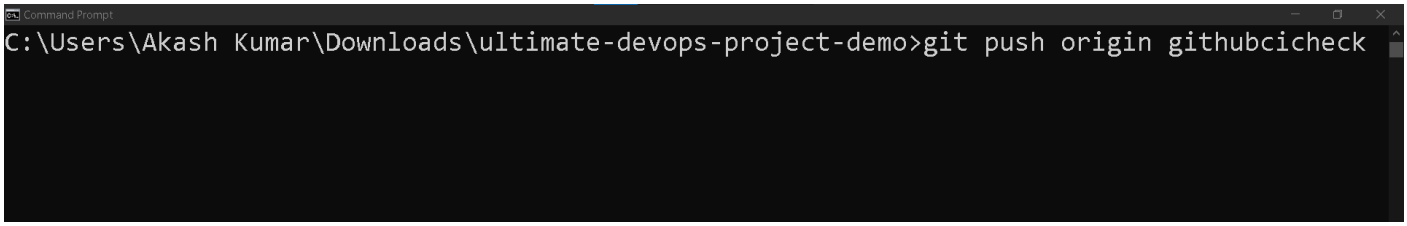
- Git checkout –b githubcicheck



- Now check status git status



- Noe git add .
- Then git commit –am “chore: verify github action”

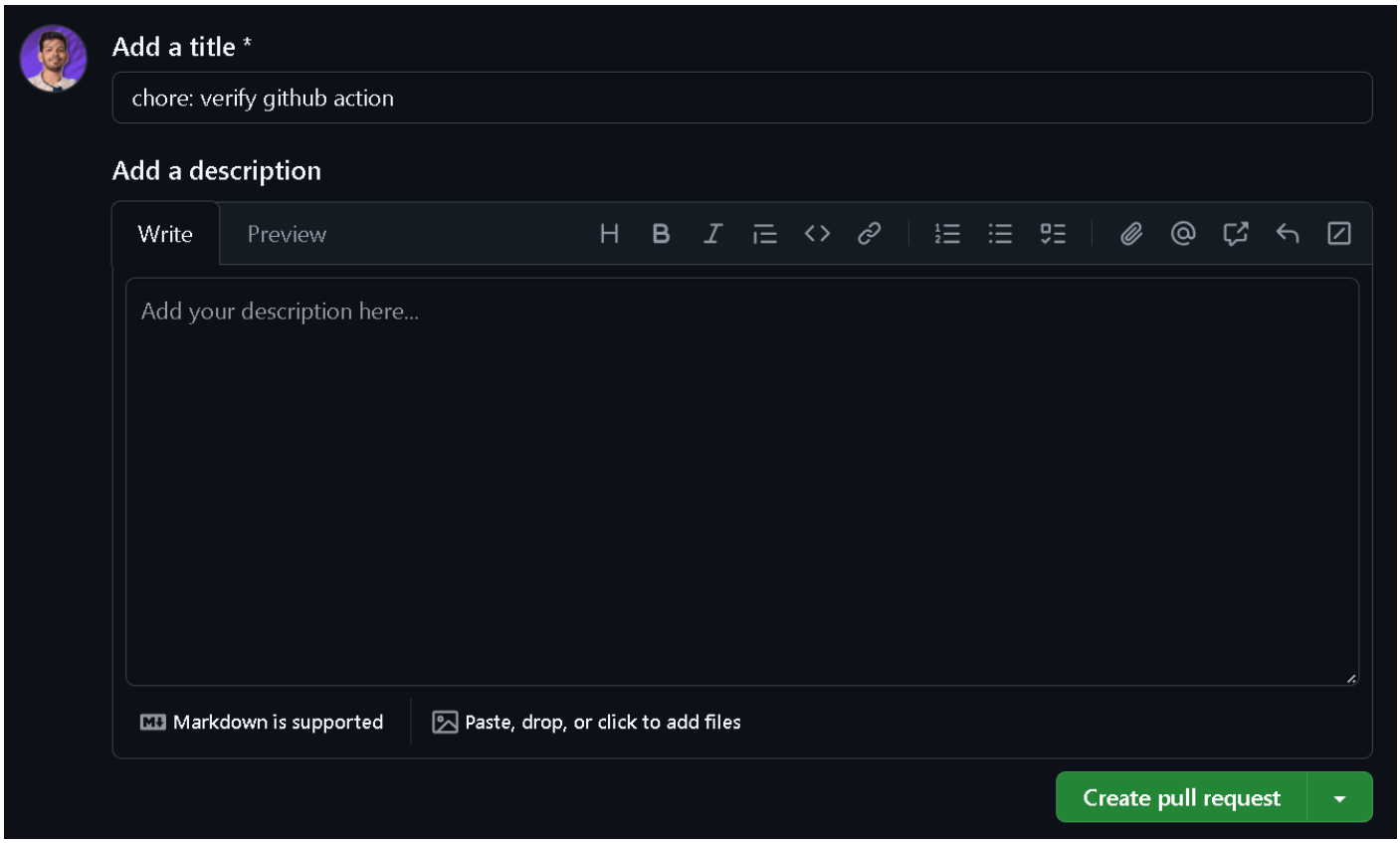


- git push origin githubcicheck

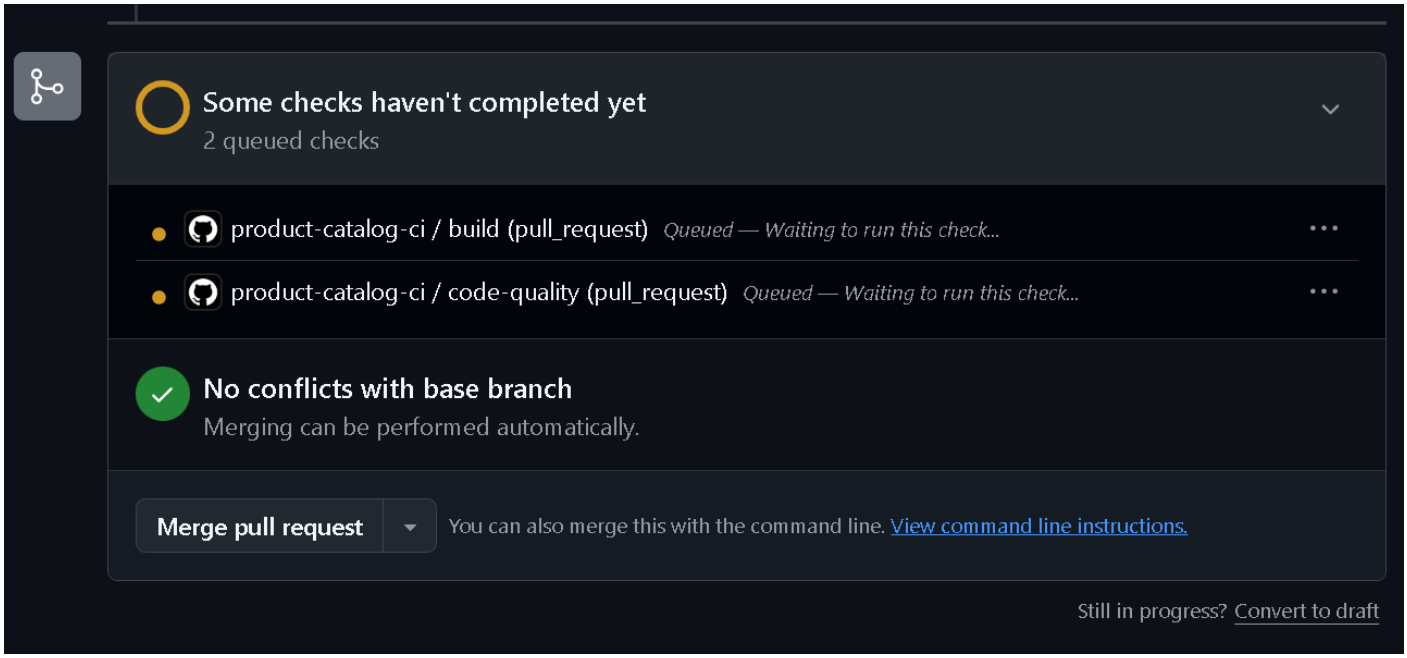
```
Command Prompt
C:\Users\Akash Kumar\Downloads\ultimate-devops-project-demo>git push origin githubcicheck
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (8/8), 606 bytes | 151.00 KiB/s, done.
Total 8 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
remote:
remote: Create a pull request for 'githubcicheck' on GitHub by visiting:
remote:      https://github.com/akashcodes-web/ultimate-devops-project-demo/pull/new/githu
bcicheck
remote:
To https://github.com/akashcodes-web/ultimate-devops-project-demo.git
 * [new branch]      githubcicheck -> githubcicheck

C:\Users\Akash Kumar\Downloads\ultimate-devops-project-demo>
```

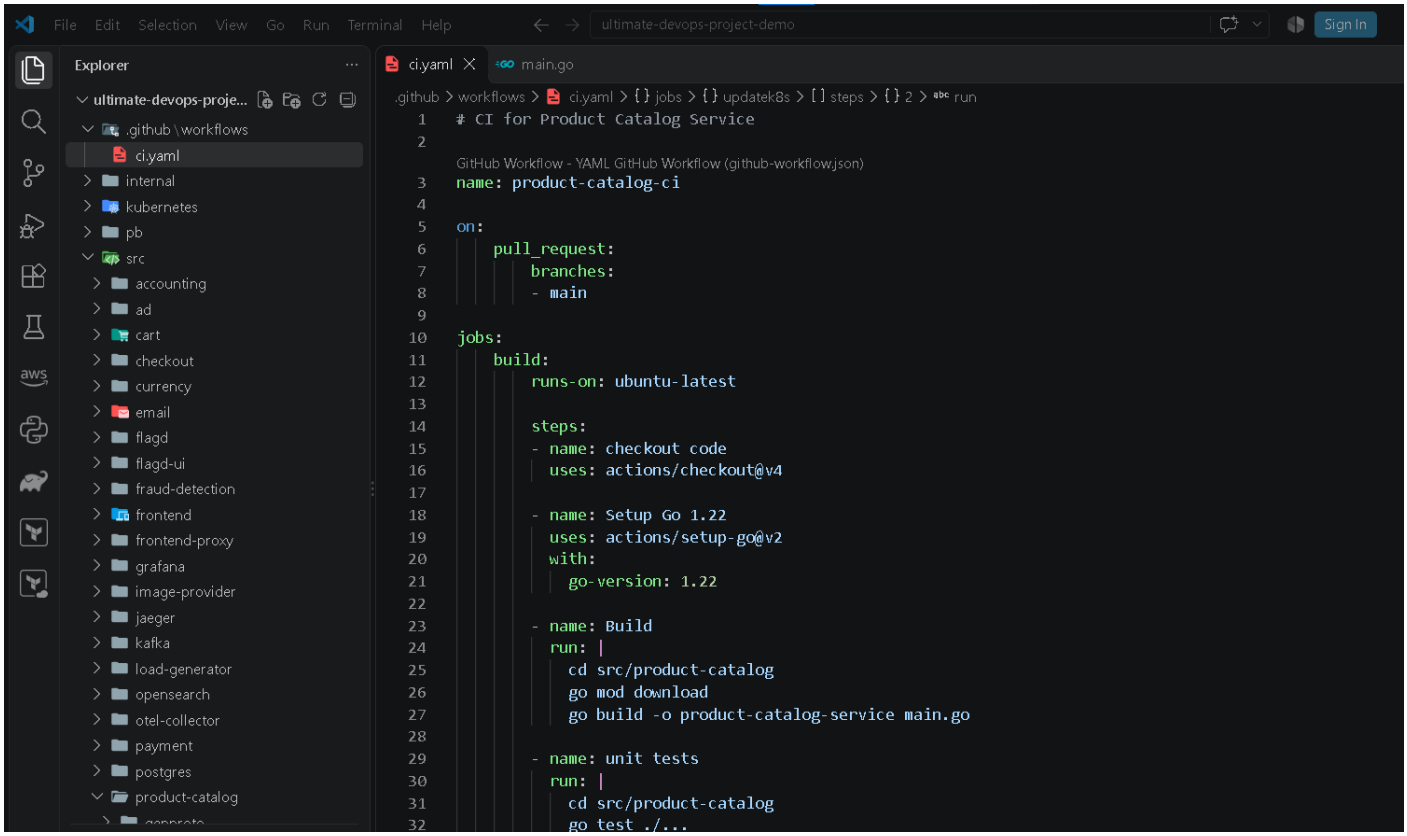
- now you have url for pull request copy it and go to browser.



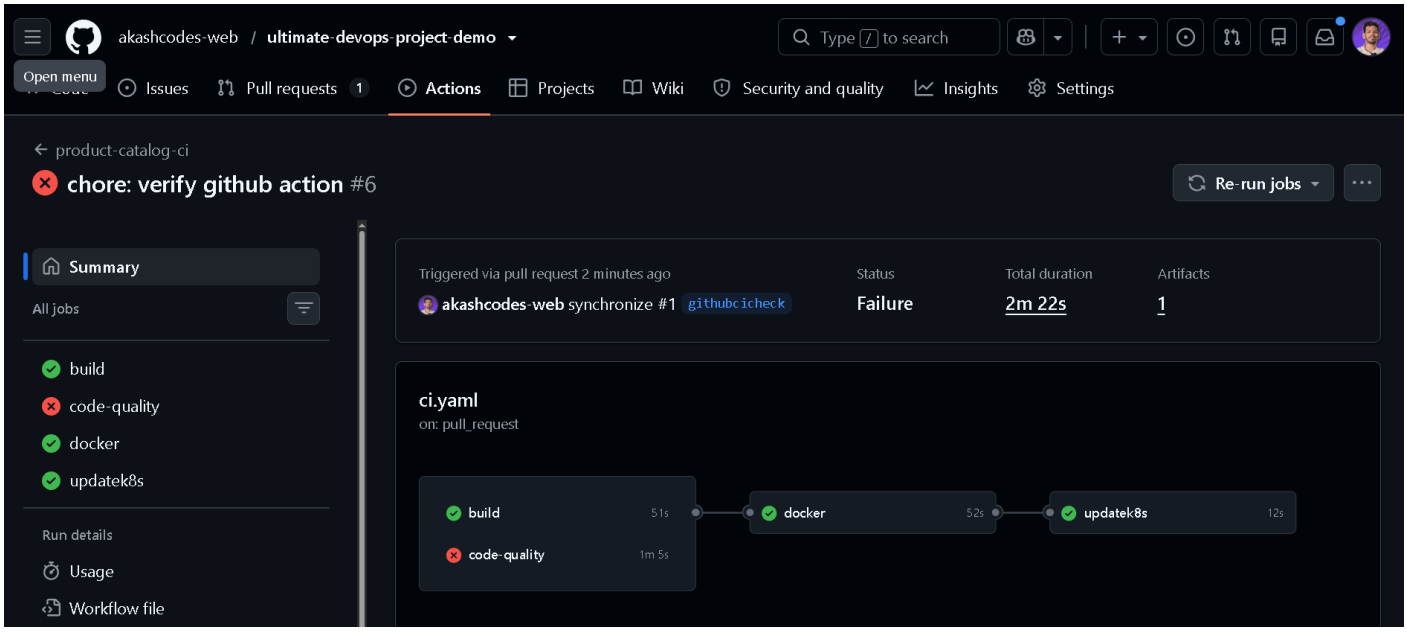
- now simply click create pull request.



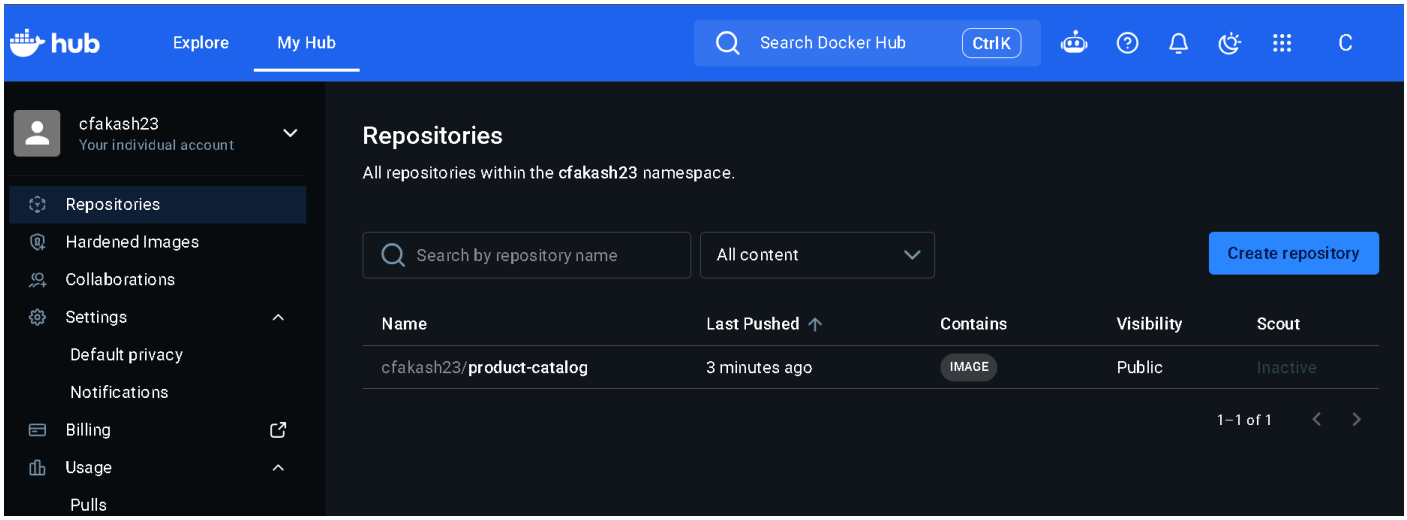
- Now our four jobs will trigger.



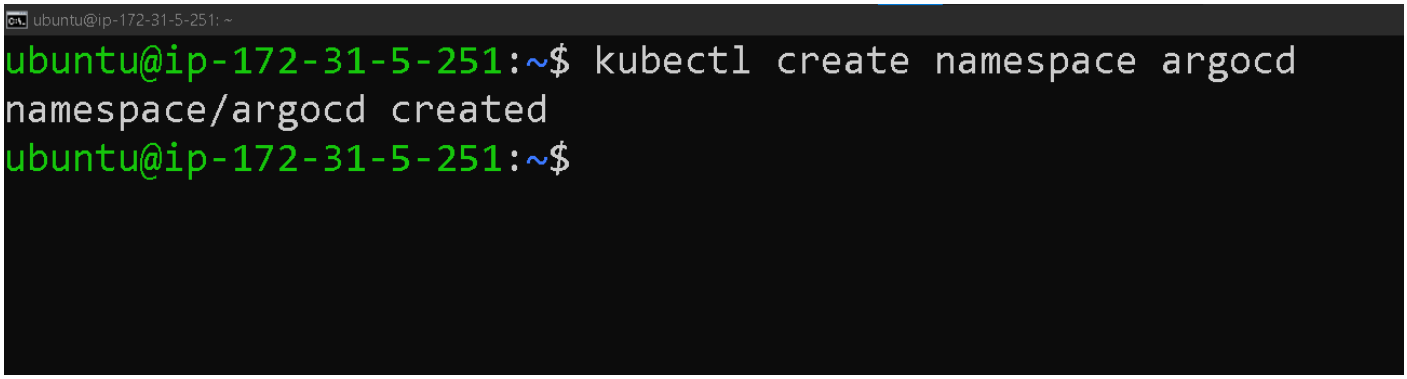
- When you go to ci.yaml you see we created 4 jobs .
- But why we see 2 in git hub
- That is because we have mentioned dependencies in updates as well as in the docker stage.
- So this will only trigger if the build stage succeeds, and this will only trigger if the Docker stage



- Now see our docker part is done.
- I kow you see some things fails this the part of developer you tell them this issue comes please resolve it.



- Now see all thing is updated in our docker.
- No go to command prompt and connect to Ec2 instance.



- kubectl create namespace argocd

```
ubuntu@ip-172-31-5-251:~$ kubectl create namespace argocd
namespace/argocd created
ubuntu@ip-172-31-5-251:~$ kubectl apply -n argocd --server-side --force-conflicts -f https
://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

- kubectl apply -n argocd --server-side --force-conflicts -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

```
ubuntu@ip-172-31-5-251:~$ kubectl get pods -n argocd
NAME                                                READY   STATUS    RESTARTS      AGE
argocd-application-controller-0                     1/1     Running   0             71s
argocd-applicationset-controller-76c75bf95b-bhcjs   1/1     Running   0             72s
argocd-dex-server-7bdc5d97c4-dbfcg                  1/1     Running   2 (45s ago)   71s
argocd-notifications-controller-5455c76599-5s6hg    1/1     Running   0             72s
argocd-redis-84d7b4b89b-8jqcf                       1/1     Running   0             71s
argocd-repo-server-58fb68dc5c-9g5cx                 1/1     Running   0             71s
argocd-server-bf66c5754-9g7v2                       1/1     Running   0             71s
ubuntu@ip-172-31-5-251:~$
```

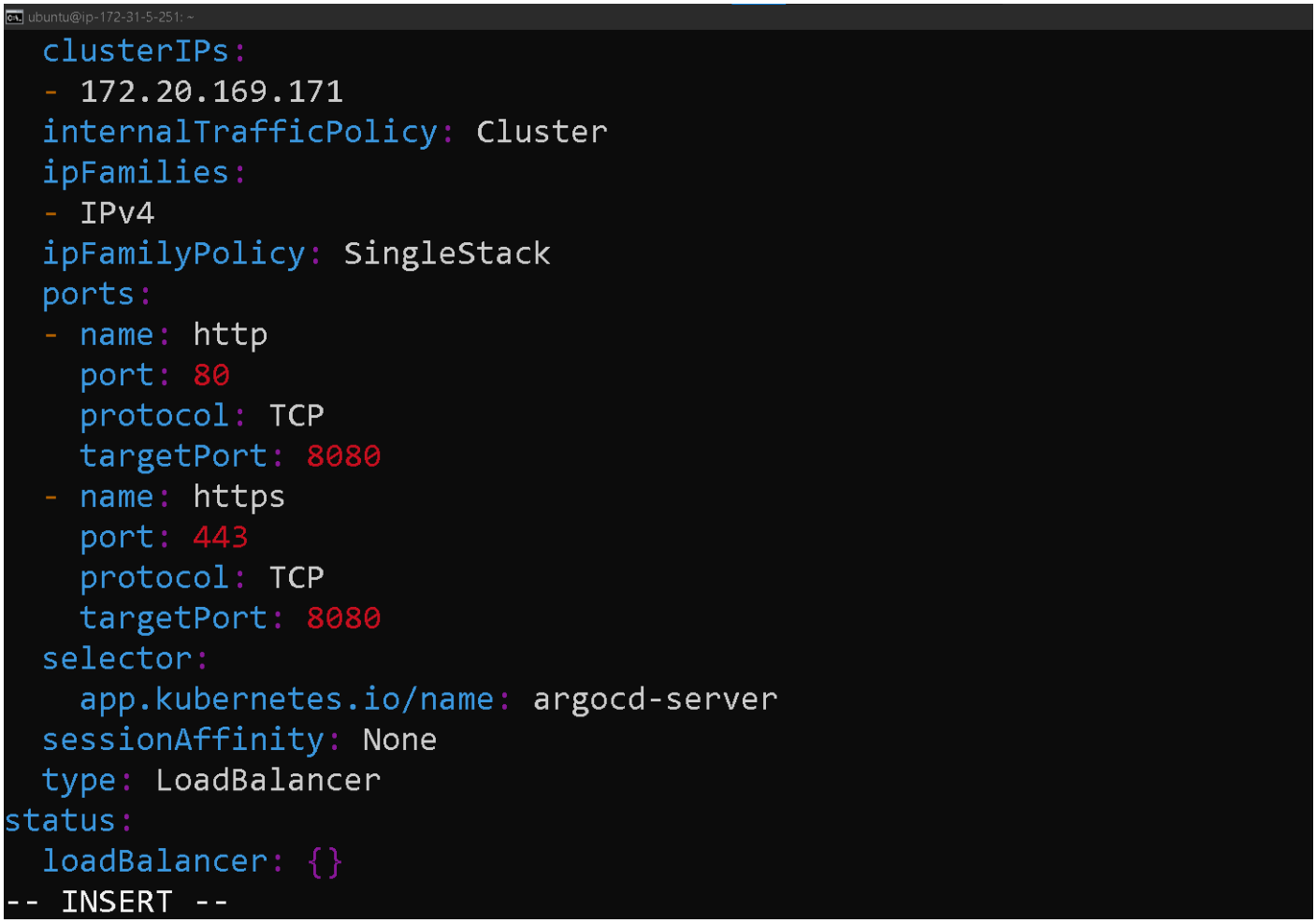
- kubectl get pods –n argocd

```
ubuntu@ip-172-31-5-251:~$ kubectl get svc -n argocd
NAME                                       TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(
S)                    AGE
argocd-application-controller              ClusterIP   None             <none>        <none
>                     2m50s
argocd-applicationset-controller           ClusterIP   172.20.61.218    <none>        7000/
TCP,8080/TCP          2m49s
argocd-dex-server                          ClusterIP   172.20.125.56    <none>        5556/
TCP,5557/TCP,5558/TCP 2m49s
argocd-metrics                             ClusterIP   172.20.35.255    <none>        8082/
TCP                   2m49s
argocd-notifications-controller-metrics    ClusterIP   172.20.31.36     <none>        9001/
TCP                   2m49s
argocd-redis                               ClusterIP   172.20.81.48     <none>        6379/
TCP                   2m49s
argocd-repo-server                         ClusterIP   172.20.254.20    <none>        8081/
TCP,8084/TCP          2m49s
argocd-server                              ClusterIP   172.20.169.171   <none>        80/TC
P,443/TCP             2m49s
argocd-server-metrics                      ClusterIP   172.20.64.73     <none>        8083/
TCP                   2m49s
```

- kubectl get svc –n argocd
- now we need argocd-server

```
argocd-server-metrics                      ClusterIP   172.20.64.73     <none>        8083/
TCP                   2m49s
ubuntu@ip-172-31-5-251:~$ kubectl edit svc argocd-server -n argocd
```

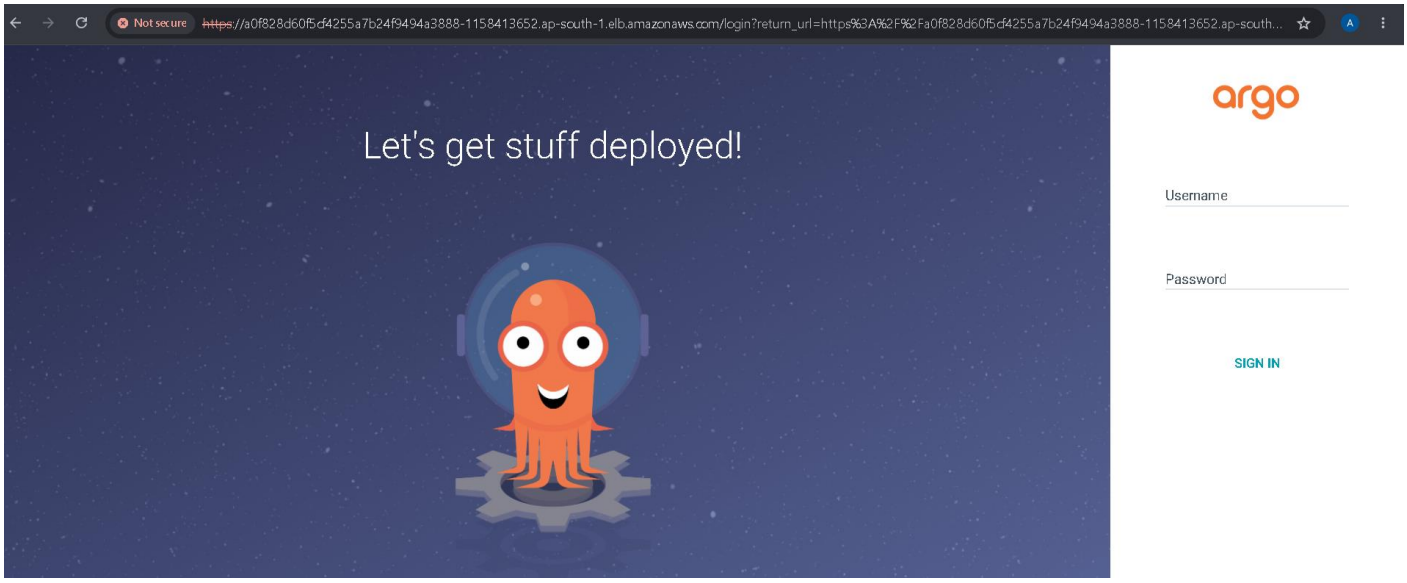
- kubectl edit svc argocd-server -n argocd



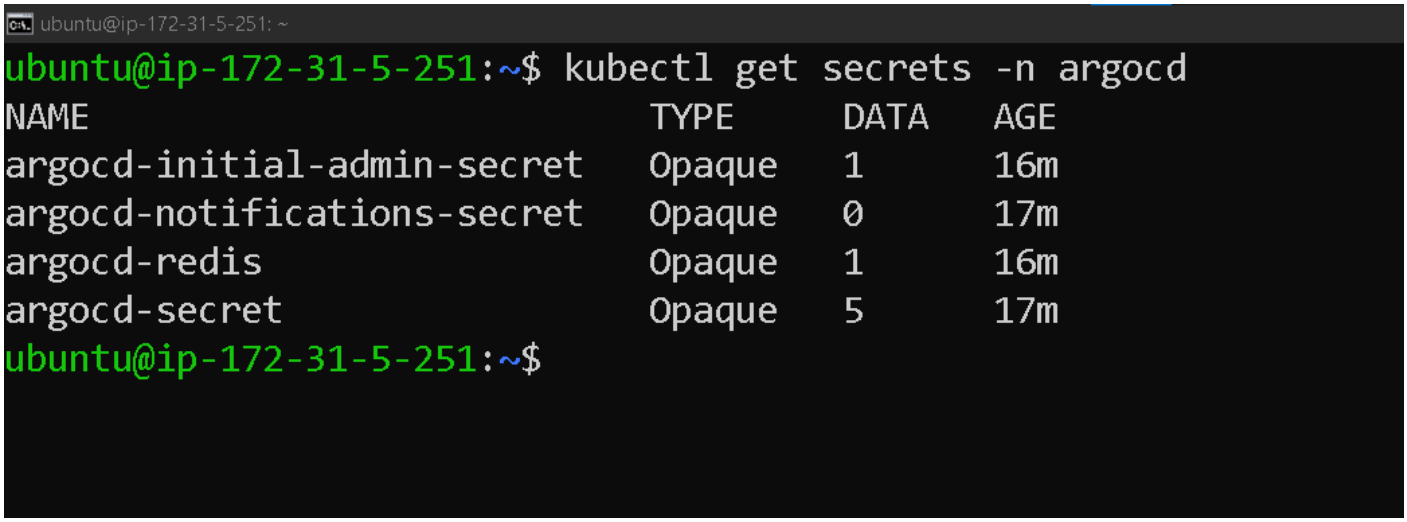
- Now change type to clustered to LoadBalancer
- Then press esc and then type :wq! and press Enter

```
ubuntu@ip-172-31-5-251: ~
ubuntu@ip-172-31-5-251:~$ kubectl get svc -n argocd
NAME                                  TYPE           CLUSTER-IP       EXTERNAL-IP
                                                       PORT(S)                      AG
E
argocd-application-controller         ClusterIP      None             <none>
                                                       <none>                       12
m
argocd-applicationset-controller      ClusterIP      172.20.61.218    <none>
                                                       7000/TCP,8080/TCP            12
m
argocd-dex-server                     ClusterIP      172.20.125.56    <none>
                                                       5556/TCP,5557/TCP,5558/TCP   12
m
argocd-metrics                        ClusterIP      172.20.35.255    <none>
                                                       8082/TCP                     12
m
argocd-notifications-controller-metrics   ClusterIP  172.20.31.36     <none>
                                                       9001/TCP                     12
m
argocd-redis                          ClusterIP      172.20.81.48     <none>
                                                       6379/TCP                     12
m
argocd-repo-server                    ClusterIP      172.20.254.20    <none>
                                                       8081/TCP,8084/TCP            12
m
argocd-server                         LoadBalancer   172.20.169.171   a0f828d60f5cf425
5a7b24f9494a3888-1158413652.ap-south-1.elb.amazonaws.com   80:32031/TCP,443:31463/TCP   12
```

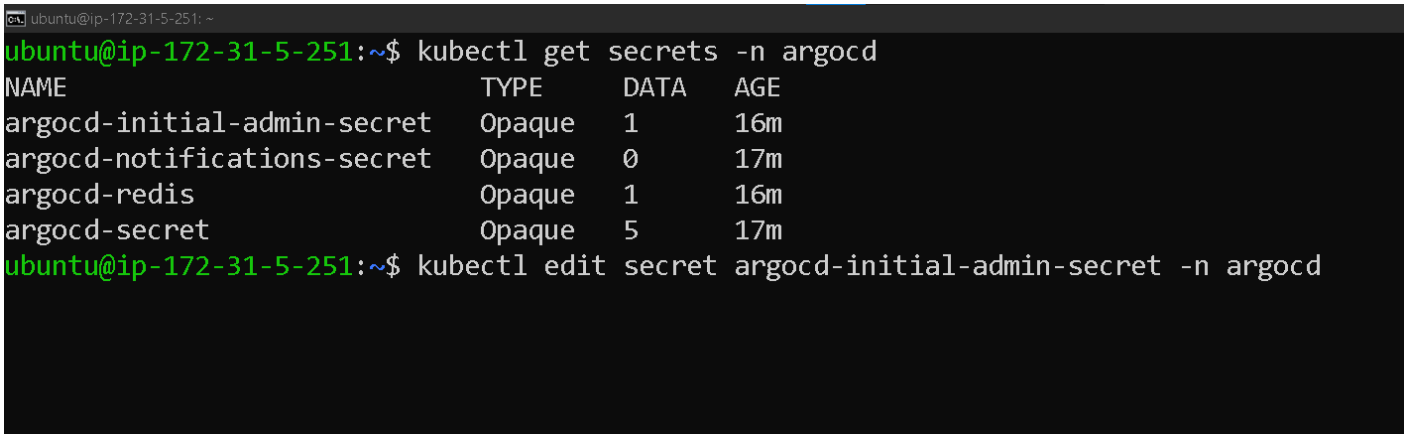
- kubectl get svc –n argocd
- now see in argocd-server load balancer is added.
- Now copy and open in the browser



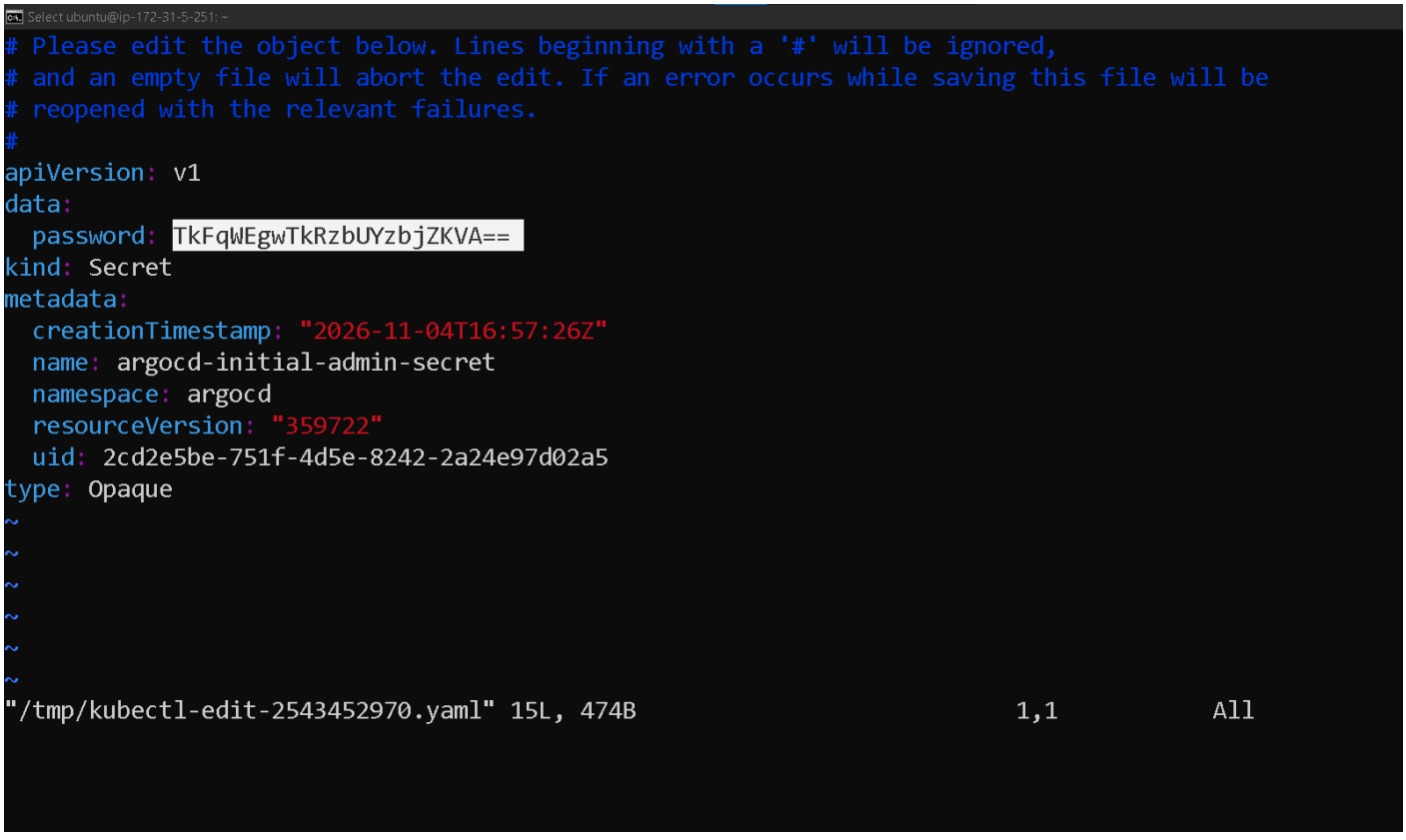
- Now see we are on argo panel.
- Now go back to control panel.



- Kubectl get secrets –n argocd
- Now we need to edit argocd-initial-admin-secret



- kubectl edit secret argocd-initial-admin-secret -n argocd



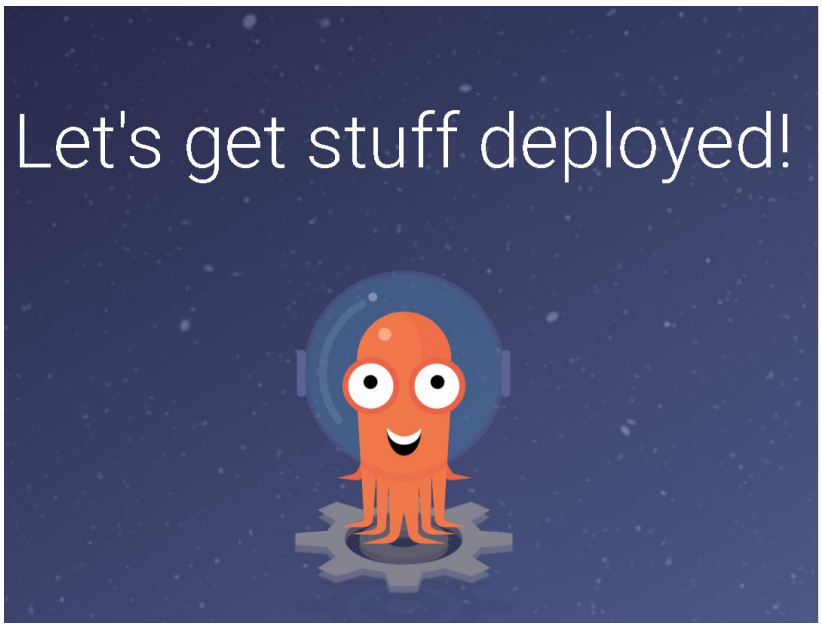
- then copy the password

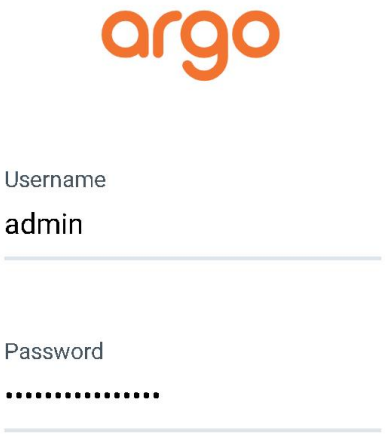
```
ubuntu@ip-172-31-5-251: ~
ubuntu@ip-172-31-5-251:~$ kubectl get secrets -n argocd
NAME                          TYPE     DATA   AGE
argocd-initial-admin-secret   Opaque   1      16m
argocd-notifications-secret   Opaque   0      17m
argocd-redis                  Opaque   1      16m
argocd-secret                 Opaque   5      17m
ubuntu@ip-172-31-5-251:~$ kubectl edit secret argocd-initial-admin-secret -n argocd
Edit cancelled, no changes made.
ubuntu@ip-172-31-5-251:~$ echo TkFqWEgwTkRzbUYzbjZKVA== | base64 --decode
```

- echo TkFqWEgwTkRzbUYzbjZKVA== | base64 –decode
- now we need to decode password

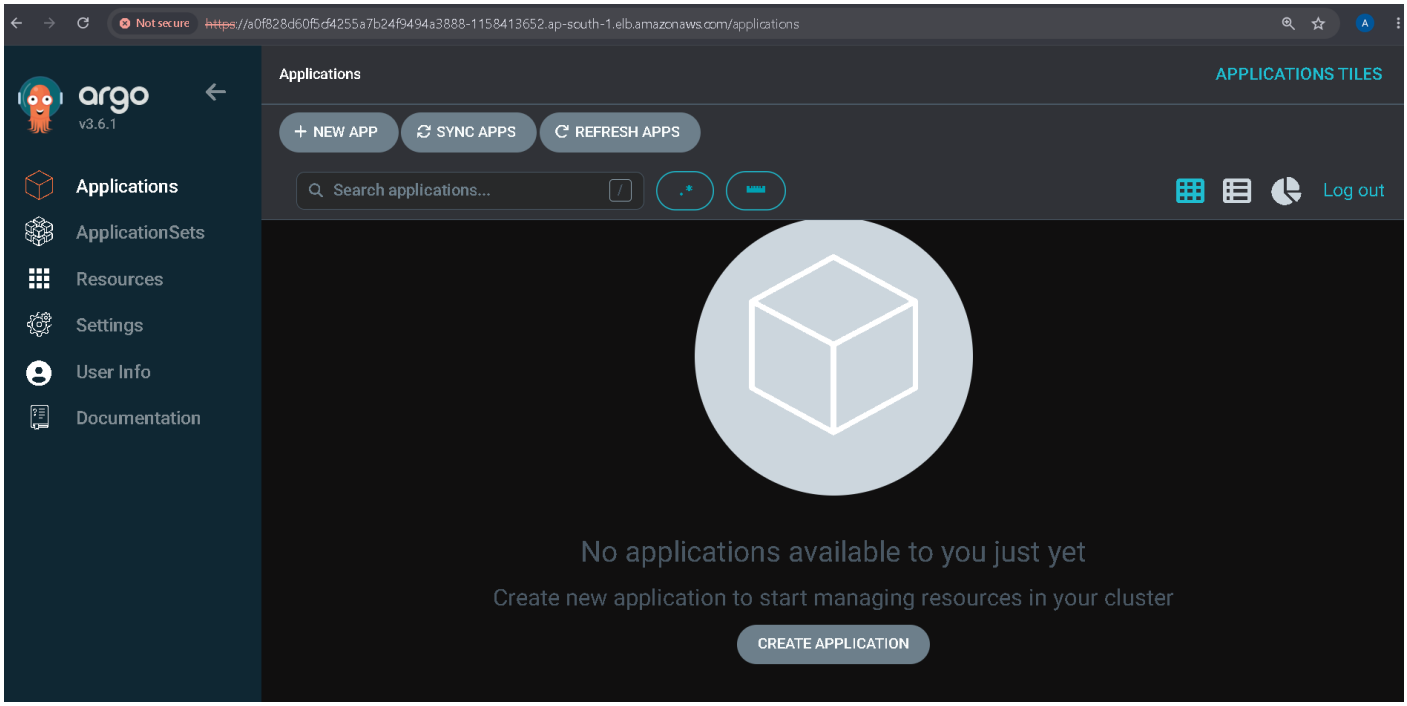
```
ubuntu@ip-172-31-5-251: ~
ubuntu@ip-172-31-5-251:~$ kubectl get secrets -n argocd
NAME                          TYPE     DATA   AGE
argocd-initial-admin-secret   Opaque   1      16m
argocd-notifications-secret   Opaque   0      17m
argocd-redis                  Opaque   1      16m
argocd-secret                 Opaque   5      17m
ubuntu@ip-172-31-5-251:~$ kubectl edit secret argocd-initial-admin-secret -n argocd
Edit cancelled, no changes made.
ubuntu@ip-172-31-5-251:~$ echo TkFqWEgwTkRzbUYzbjZKVA== | base64 --decode
NAjXH0NDsmF3n6JTubuntu@ip-172-31-5-251:~$
```

- now before Ubuntu all are password

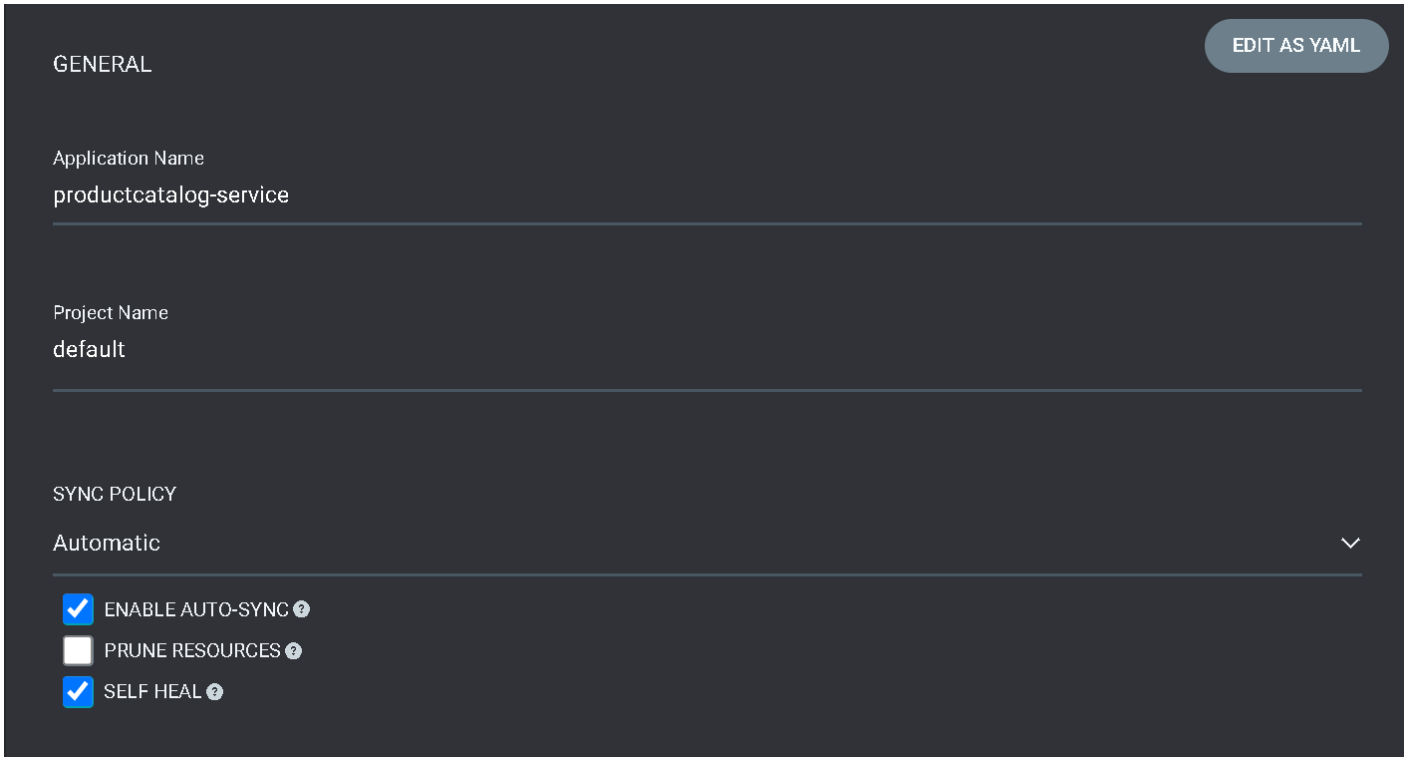




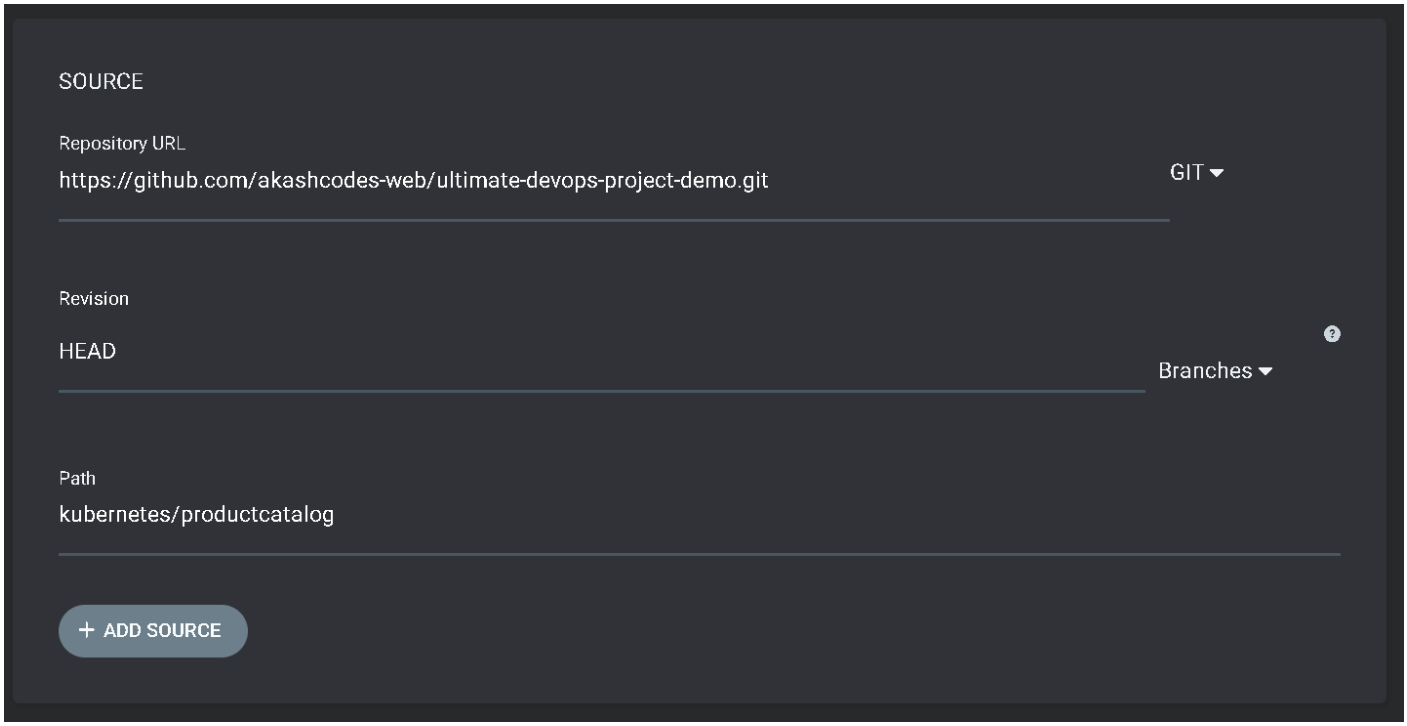
- now username is admin
- and password that you get paste it.



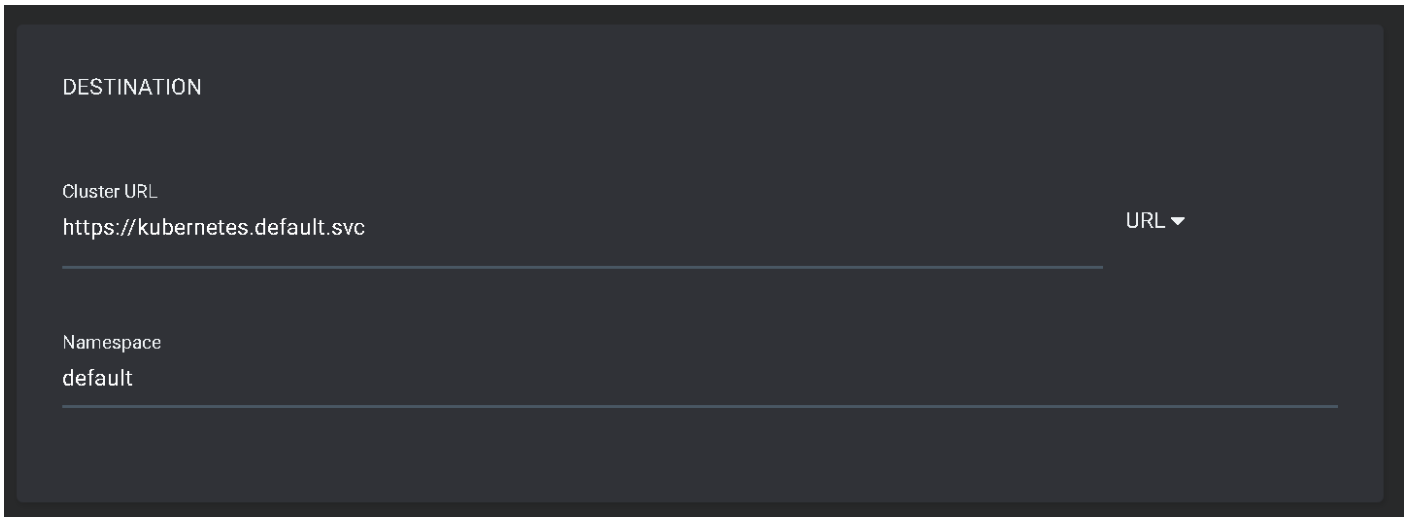
- Now we are in our argo console.
- Now click create Application



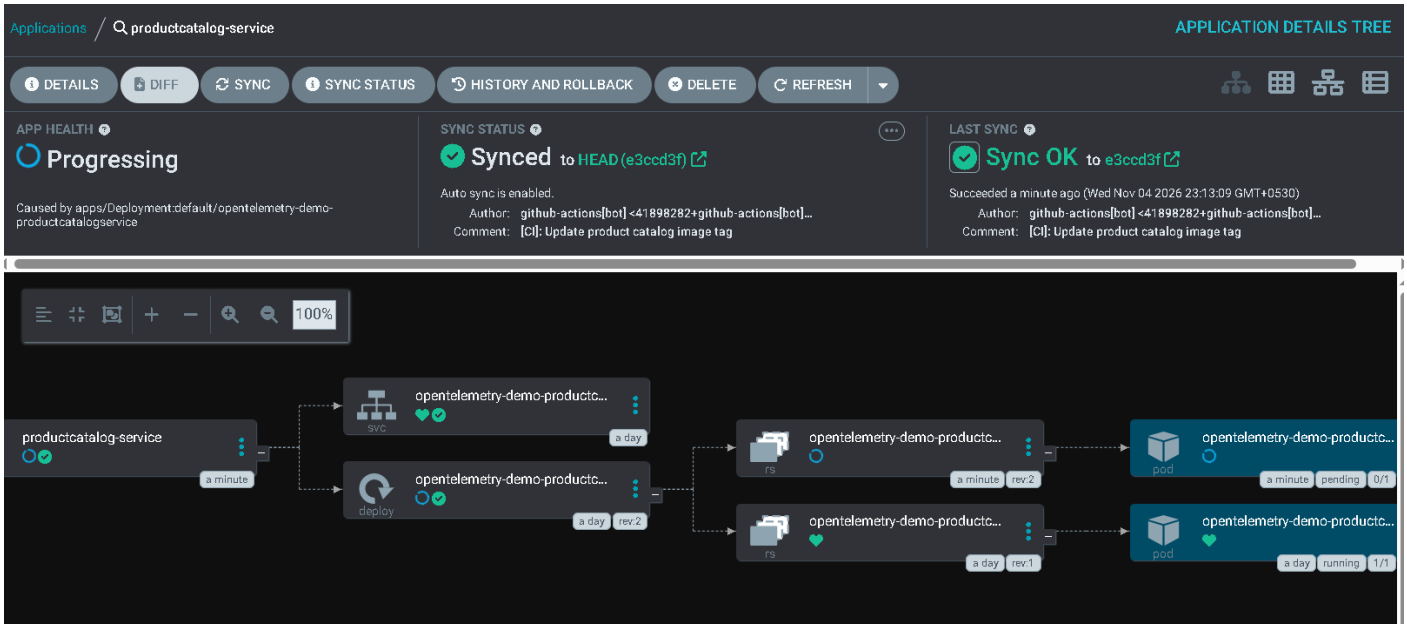
- Now name productcatalog-service



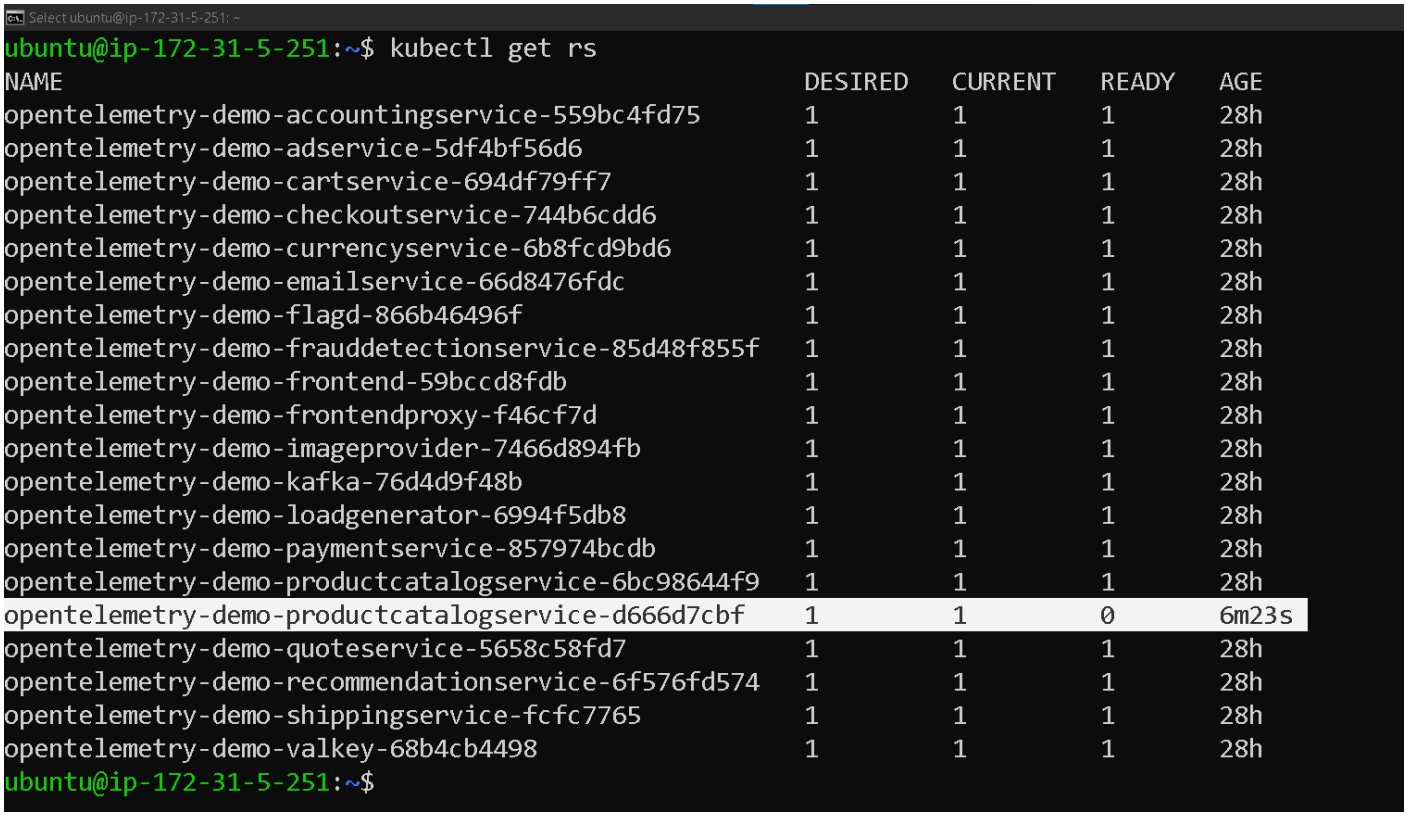
- Now scroll down in source paste git repo
- And in path kubernetes/productcatalog



- Now select cluster URL
- And then namespace default
- And simply click create



- Now its done and wait some time



- kubectl get rs
- now see its updated

```
ubuntu@ip-172-31-5-251: ~
          value: "8080"
        - name: FLAGD_HOST
          value: opentelemetry-demo-flagd
        - name: FLAGD_PORT
          value: "8013"
        - name: OTEL_EXPORTER_OTLP_ENDPOINT
          value: http://$(OTEL_COLLECTOR_NAME):4317
        - name: OTEL_RESOURCE_ATTRIBUTES
          value: service.name=$(OTEL_SERVICE_NAME),service.namespace=opentelemetry-demo,se
rvice.version=1.12.0
        image: cfakash23/product-catalog:41869594313
        imagePullPolicy: IfNotPresent
        name: productcatalogservice
        ports:
        - containerPort: 8080
          name: service
          protocol: TCP
        resources:
          limits:
            memory: 20Mi
        terminationMessagePath: /dev/termination-log
/image
```

- Now see images is updated